

北京邮电大学高等教育自学考试

毕 业 设 计

论文题目：书云网在线图书限时借阅系统
设计与实现

姓 名 _____

准 考 证 号 _____

专 业 计算机科学与技术

指 导 教 师 _____

完成时间 2024 年 3 月 10 日

北京邮电大学高等教育自学考试 本科毕业设计（论文）诚信声明

本人声明所呈交的毕业设计（论文），题目《书云网在线图书限时借阅系统设计与实现》是本人独立进行研究工作所取得的成果。尽我所知，除了文中特别加以标注和致谢中所罗列的内容以外，论文中不包含其他人已经发表或撰写过的研究成果，也不包含为获得北京邮电大学高等教育自学考试或其他教育机构的学位或证书而使用过的材料。

论文与资料若有不实之处，本人承担一切相关责任。

本人签名：



日期： 2024.03.10

查重报告



简洁报告-大学生版

基础信息

论文标题: 一书云网在线图书限时借阅系统设计与实现
作者姓名: 所属单位: 北京邮电大学自学考试
报告编号: 检测时间: 2024-03-10 19:01:36
检测字符数: 25601

检测结果

全文总相似比: **3.27%** (全文总相似比 = 复写率 + 他引率 + 自引率 + 专业术语)
复写率: 3.05% 他引率: 0.22% 自引率: 0.0%
专业术语: 0.0% 自写率: 96.73%

相似片段

总相似片段数: 14
期刊: 0 博硕: 6 综合: 0
外文: 0 自建库: 0 互联网: 8

检测范围

中文科技期刊论文全文数据库	中文主要报纸全文数据库	中国专利特色数据库
博士/硕士学位论文全文数据库	中国主要会议论文特色数据库	港澳台文献资源
外文特色文献数据全库	维普优先出版论文全文数据库	互联网数据资源/互联网文档资源
高校自建资源库	图书资源	古籍文献资源
个人自建资源库	年鉴资源	IPUB原创作品

时间范围: 1989-01-01至 2024-03-10

指标说明:

复写率: 相似或疑似重复内容占全文的比重
他引率: 引用他人的部分占全文的比重, 请正确标注引用
自引率: 引用自己已发表部分占全文的比重, 请正确标注引用
专业术语: 公式定理、法律条文、行业用语等占全文的比重

关注公众号

唯一指定官网
vpcs.cqvip.com



北京邮电大学高等教育自学考试

毕业设计（论文）任务书

姓名		准考证号		专业	计算机信息管理	
工作单位				职务岗位		
邮箱				联系电话		
自考本科 前学历	大专	毕业院校		所学专业		
设计（或论文）题目		在线图书限时借阅系统设计与实现				
本设计选题根据（欲解决的问题及其学术或实用价值）： “书籍是人类进步的阶梯”，然而随着人们的接触信息的渠道增多、信息碎片化等因素，往往导致相比于读书所接受的知识水平的大幅下降。就算打开图书也很容易虎头蛇尾，只看了个开头，学习效果大打折扣。 以上为背景，早在清朝时袁枚在《黄生借书说》一文中就给出过答案——“书非借不能读也”。基于“借”的思路，开发设计一款基于 Web 技术的在线图书限时借阅系统。以“借”、“限时”和“限量”来提升读者的紧迫感，使其能“惴惴焉摩玩之不己”、“读书也必专”，从而达到提高读书效率、提升个人素养的目标。						
本设计的主要研究内容（设计要求和设计步骤要求详细到章节）： 第 1 章 绪论 1.1 研究的背景及意义 1.2 相关开发技术介绍 第 2 章 需求分析 2.1 设计目标 2.2 功能需求分析 第 3 章 逻辑设计 3.1 系统功能设计 3.2 数据库设计 第 4 章 详细设计 4.1 界面设计 4.2 模块设计 第 5 章 系统实现 5.1 模块关键代码 5.2 系统部署运行						

第 6 章 系统测试 6.1 测试的目的与内容 6.2 功能测试与结果 第 7 章 总结 参考文献 致谢	
本人在该设计中完成的具体工作： 本人独立完成该设计	
主要参考文献、资料： [1] Bruce Eckel,陈昊鹏.Java 编程思想（第 4 版）[M].北京:机械工业出版社,2007.6 [2] 刘世峰.信息系统开发与管理[M].北京:机械工业出版社,2018.8 [3] 黄靖.数据库系统原理[M].北京:机械工业出版社,2018.4 [4] 肖文吉.Spring+MyBatis 企业应用实战[M].北京:电子工业出版社,2017.1 [5] Ben Forta,刘晓霞,钟鸣.MySQL 必知必会[M].北京:人民邮电出版社,2009.1 [6] 杨开振.深入浅出 MyBatis 技术原理与实战[M].北京:电子工业出版社,2016.9 [7] 鸟哥.鸟哥的 Linux 私房菜:基础学习篇 4 版[M].北京:人民邮电出版社,2018.11 [8] 朱树敏.大数据背景下基层公共图书馆数字阅读推广研究[J].图书馆工作与研究,2022	
审批意见	教师签字：

任务书下达时间（教师填写） 年 月 日

注：本表格式可调整，填写内容不要超过 2 页，完成后 A4 纸双面打印，附在论文前。

书云网在线图书限时借阅系统设计与实现

摘 要

在互联网高普及率的当代，人们的娱乐方式发展多元化，读书的人也逐年在减少，读书群体对书的接受形式也发了重大改变，选择看电子书而不是纸书的人越来越多。在这个纸书阅读越发减少的时代下，书的形式也要适当地扩展到线上，由于信息大爆炸以及接受信息的碎片化因素，读者往往会迷失在资讯、广告等分散精力的无用信息中，看书更容易出现虎头蛇尾，醒悟时难免愧疚难当与自责，读书效果、学习水平、思考深度都大打折扣。

是时候改变这个现实状况了，本人受清代读书人袁枚的“书非借不能读”影响，产生设计与开发一个界面不花哨、功能实用和有效率的在线阅读平台，借用“借”的想法将在线阅读平台改进为在线图书限时借阅平台，对系统名称进行选择最终定名为“书云网在线图书限时借阅平台”（简称，书云网图书借阅平台）。

本人在互联网大环境下，对广大读者的现状使用软件工程的分析设计流程，对读者群体进行需求分析、业务建模，将业务建模转化为数据库系统的设计，并使用开源开发技术对以上分析的需求及数据模型转换为源代码，实现功能需求并对测试用例一一执行，改进编码过程中的缺陷与不足，最终完成整个设计。本篇论文即是对书云网图书借阅平台的完整分析、设计与实现说明书。

关键词 图书借阅平台 开源技术 软件设计

Design and implementation of the ShuYunwang online books limited time borrowing system

ABSTRACT

In the contemporary era of high Internet popularity, people's entertainment methods are diversified, and the number of people reading books is also decreasing year by year. The acceptance form of books by reading groups has also changed significantly, and more and more people choose to read e-books instead of paper books. In this era of less and less paper book reading, the form of books should also be appropriately extended to online. Due to the explosion of information and the fragmentation of receiving information, readers tend to get lost in information, advertising and other useless information that distracts their energy. Reading books is more likely to have an end, and it is hard to avoid guilt and self-blame when they wake up. Reading effect, learning level, depth of thinking are greatly reduced.

It is time to change this reality. Influenced by the Qing Dynasty scholar Yuan Mei's "books must be borrowed and cannot be read", I have designed and developed an online reading platform with non-colorful interface and practical and efficient functions. Borrowing the idea of "borrowing", I have improved the online reading platform into an online book limited time borrowing platform. The system name was selected and finally named as "Shuyunnet online book Limited Time Lending platform" (referred to as Shuyunnet book lending platform).

In the Internet environment, I use the analysis and design process of software engineering to analyze the current situation of the majority of readers, analyze the needs of the reader group, model the business, and convert the business modeling into the design of the database system, and use the open source development technology to convert the above analysis requirements and data model into source code, realize the functional requirements and execute the test cases one by one. Improve the defects and deficiencies in the coding process, and finally complete the whole design. This paper is a complete analysis, design and implementation of the book cloud network library platform.

KEY WORDS book lending platform open-source technology software designing

目 录

第一章 绪论	1
1.1 研究的背景及意义	1
1.2 开发目标	1
1.3 相关开发技术介绍	1
1.3.1 B/S 架构	2
1.3.2 MVC 模式	2
1.3.3 Layui 组件库	2
1.3.4 SpringBoot 后端框架	2
1.3.5 Maven	2
1.3.6 Tomcat	3
1.3.7 Minio	3
1.3.8 MySQL	3
第二章 需求分析	3
2.1 设计目标	3
2.2 功能需求分析	4
2.2.1 背景信息	4
2.2.2 目标用户	4
2.2.3 行业背景	4
2.2.4 功能需求	4
2.2.5 数据安全需求	4
2.2.6 用户界面需求	5
2.2.7 需求优先级分析	5
2.3 业务分析	5
2.3.1 数据流图	5
2.3.2 数据字典	7
2.4 可行性分析	12
2.4.1 经济可行性	12
2.4.2 技术可行性	12
2.4.3 管理可行性	13
2.4.4 开发环境可行性	13
2.4.5 法律可行性	13
2.4.6 使用可行性	13

2.4.7 可行性分析结论	13
第三章 概要分析	14
3.1 架构设计	14
3.1.1 软件架构设计	14
3.1.2 部署架构设计	15
3.2 数据库设计	16
3.2.1 数据库概念设计.....	16
3.2.2 数据表逻辑设计.....	18
3.2.3 数据表物理设计.....	19
3.3 模块设计.....	20
3.4 系统功能设计	21
3.4.1 用户管理模块功能.....	21
3.4.2 分类管理模块功能.....	21
3.4.3 图书管理模块功能.....	22
3.4.4 借阅管理模块功能.....	22
3.4.5 收藏管理模块功能.....	23
第四章 详细设计.....	23
4.1 界面设计.....	23
4.1.1 首页.....	23
4.1.2 用户注册页.....	24
4.1.3 用户登录页.....	25
4.1.4 图书搜索结果页.....	25
4.1.5 图书详情页.....	26
4.1.6 图书阅读页.....	26
4.1.7 图书管理页.....	27
4.1.8 图书分类管理页.....	27
4.1.9 借阅管理页.....	27
4.1.10 收藏管理页	28
4.1.11 关于本站页	28
4.2 逻辑设计.....	29
4.2.1 用户管理模块功能逻辑.....	29
4.2.2 分类管理模块功能逻辑.....	32
4.2.3 图书管理模块功能逻辑.....	33
4.2.4 借阅管理模块功能逻辑.....	36
4.2.5 收藏管理模块功能逻辑.....	38

第五章 系统实现	40
5.1 开发语言的选择	40
5.2 软件开发工具的选择	40
5.3 模块关键代码	41
5.3.1 用户管理模块	41
5.3.2 分类管理模块	42
5.3.3 图书管理模块	43
5.3.4 借阅管理模块	44
5.3.5 收藏管理模块	45
5.4 系统部署运行	46
5.4.1 部署环境准备	46
5.4.2 安装 MinIO 服务端	46
5.4.3 安装 MySQL 数据库	47
5.4.4 编译项目源代码	48
5.4.5 启动项目	48
5.5 用户界面展示	48
第六章 系统测试	51
6.1 测试的目的与内容	51
6.2 功能测试与结果	51
6.2.1 用户管理模块功能测试	51
6.2.2 分类管理模块功能测试	52
6.2.3 图书管理模块功能测试	54
6.2.4 借阅管理模块功能测试	54
6.2.5 收藏管理模块功能测试	55
第七章 总结	56
参考文献	57
致谢	58
附录	59

第一章 绪论

1.1 研究的背景及意义

阅读自古以来都是有其进步意义的，无论是在线阅读还是传统纸质阅读方式，人们无法忽略它的现实意义的；阅读是一种受科学技术、政治经济等因素的影响而不断变化的社会文化^[1]。但在互联网高普及率的当代，人们的娱乐方式发展多元化，读书的人也逐年在减少，读书群体对书的接受形式也发了重大改变：选择看电子书而不是纸书的人越来越多。在这个纸书阅读越发减少的时代大背景下，书的形式更应扩展到线上，由于信息的碎片化因素，读者往往会迷失在资讯、广告等分散精力的无用信息中，看书更容易出现虎头蛇尾的现象，读书效果、学习水平和思考深度都大打折扣。

对于以上背景，考虑到本人已接受了计算机专业的系统性学习，可以使用软件工程的结构化方法，来发掘读者用户需求，进行业务流程分析和数据库设计，以及使用开源技术实现图书借阅系统。在软件测试的过程中发现程序错误，并予以改正，提高代码的鲁棒性和系统稳定性。

开发和设计这样一个在线的图书借阅系统有其积极意义：读者使用此系统能提高阅读效率、阅读更专注。阅读众多典籍能开拓人生观、世界观，改变对事物的观点，真正实现腹有诗书气自华。

1.2 开发目标

本系统的开发目标是：通过充分了解用户需求，根据功能需求和逻辑流程对系统进行结构化分析，设计与描述系统的运行模式，基于设计方案实现一款界面简单清爽、能实现在线阅读、利用“借阅”的心理感受，能提高用户阅读效率的书云网在线图书限时借阅系统（简称，书云网图书借阅系统）。

1.3 相关开发技术介绍

本系统预计基于 B/S 架构的 Web 项目，整体结构分为前端和基于 MVC 模式的后端。前端使用 HTML+CSS+JavaScript+Layui；后端使用 Java 语言开发，基于 Spring Boot 框架实现。最终使用 Maven 构建打包成 War 包，使用内嵌 Tomcat 运行项目。图书封面和电子书文件使用 MinIO 对象存储。后端程序通过 JDBC 驱动与 MySQL 进行交互持久化应用数据。项目程序、MinIO、MySQL 数据库均运行在 Linux 服务器中。

1.3.1 B/S 架构

B/S 架构是以浏览器作为 Client 端，以服务器作为 Server 端的架构模式。使用该架构原因在于操作系统往往会预装浏览器，用户无需 PC 安装客户端，也能通过浏览器获得多端一致的使用体验。可以通过符合 W3C 标准的 Web 技术(如 HTML、CSS、JavaScript、Ajax 等)快速开发 Web 系统，遂选用此 B/S 架构作为本系统的应用架构。

1.3.2 MVC 模式

MVC (Model/View/Controller)，即数据模型、界面和控制器模式。该模式使用分层的思想，使数据模型、界面和控制器相分离实现互不影响。其中：Model 对数据进行渲染结构，View 对界面绘制，Controller 控制业务流程。可以使用自顶向下的结构化设计思想，依次实现各层功能，最终组合成完整的功能。

可见，使用 MVC 模式可以降低软件系统的复杂度、耦合度，遂使用 MVC 模式实现本系统。

1.3.3 Layui 组件库

Layui 组件库是一组国人开源的免费用户界面组件库，轻量级是它的优势之一。以 HTML/CSS/JavaScript 为基本的开发模式，使前端新手也可快速上手。组件美观度很高并且官方文档准确完备，非常适合网页界面的快速构建。遂使用该 UI 组件库实现本系统。

1.3.4 Spring Boot 后端框架

Spring Boot 脱胎于开源社区的 Spring 框架，Spring 提供了对象生命周期管理、AOP 切面编程、事件驱动 (Event Driven)、依赖反转 (DI) 等高级特性，在此基础上 Spring Boot 传承了 Spring 的设计哲学——约定优先于配置，使集成开源组件更加简易，用户从繁杂的配置中解放出来，引入需要的组件 starter 即可完成集成进入项目。开发人员可以更好地关注业务功能的实现，从而提高开发效率。遂使用该后端框架实现本系统。

1.3.5 Maven

Maven 是 Apache 组织开源的老牌 Java 构建工具，通过 Maven 可以快速地将开源组件引入到项目依赖树中。可经由一些简单的命令或图形化操作，完成源代码构建成制品、生成 API 文档、自动化执行测试用例、发布制品到制品仓库等标准化流程。

1.3.6 Tomcat

Tomcat（全称 Apache Tomcat），是 Apache 开源组织管理的经典 Java Web 应用服务器。Tomcat 支持 Servlet API、Java Server Page（JSP）、WebSocket、Filter、WebFlux 开发标准 API，其部署模式在 Spring Boot 框架设计人员的积极推动，由原本仅支持应用 War 放到 webapps 目录中部署，衍生为支持嵌入式地运行在 War 包之中，部署维护非常简单，又因其性能稳定及项目维护积极，遂选用它作为本系统的 Web 应用服务器。

1.3.7 MinIO

MinIO 是开源对象存储服务（OSS），号称完美兼容亚马逊 S3 API 接口，并提供性能较高的能力。使用 OSS 服务可以避免将二进制大文件存储到数据库中所致的性能低下问题，以及避免将大量文件直接存储到操作系统目录中的存储空间浪费问题。使用 Java 的 MinIO 客户端对 MinIO 进行交互，实现本系统的文件上传与下载功能。

1.3.8 MySQL

MySQL 数据库是世界上最流行的开源的关系型数据库之一，分为企业版与社区版两个版本。社区版是免费的，企业版提供针对企业数据库集群环境的支持，在本项目设计中其集群能力是非必要的，所以采用社区版 MySQL 做为数据库的选型。

第二章 需求分析

2.1 设计目标

基于 Web 技术与 Java 编程语言设计与实现一款支持图书的在线搜索、在线借阅、在线阅读、限时归还功能，并集系统管理功能于一身的免费在线图书借阅平台。

此系统要有别于传统图书馆、书店，主要有三方面区别：一方面要满足用户的电子化阅读需求；第二方面要借助“借”的心理设计图书借阅与图书限量提供；第三方面要做到借阅时长到期时自动归还功能，以匹配现实世界中借的东西被索还的场景。使用户处于“真实”借书的沉浸式体验，从而真正的达到对书的精读和读书效率提升的根本目标。

2.2 功能需求分析

2.2.1 背景信息

随着信息技术的进步，人们的生活已进入信息化时代，信息碎片化、接受信息多样化，使“有深度”地读书变得难能可贵，携带纸书相对不便，准备设计一款在线图书限时借阅系统解决这个问题。

2.2.2 目标用户

本系统设计的目标用户针对于那些经常面对 PC、有想深入学习想法的上班族。这些用户都有接触计算机的使用经验，对新事物的接受能力比较强，有强烈的阅读冲动但苦于干扰太多的人。

2.2.3 行业背景

主要是商业公司和图书馆有在线图书功能。商业公司一方面提供图书线上阅读，另一方面通过付费使用方式获得收益；图书馆的线上功能一般是提供图书信息和馆藏数据库的查询功能，少见有在线阅读功能的图书馆，在线借阅的主体是真实存在的书本，形式上仍是传统纸书的借阅管理模式。以上两者几乎没有在线借阅的功能。

2.2.4 功能需求

功能需求包括：用户的自主注册和自主登录、用户搜索图书、查看图书信息、对图书进行借阅，以及对已借阅的图书进行在线阅读、自动还书等功能。

用户角色包含超级管理员、赋权管理员以及普通用户这三类。图书管理、用户管理和图书分类管理，都只能由管理员角色使用。普通用户除自主注册和登录外，允许使用自维护的收藏管理、借阅管理功能。

搜索图书功能要同时支持关键字与分类搜索功能。

2.2.5 数据安全需求

除了用户注册与登录功能外，其余接口调用均需校验用户登录态。仅管理员可用的接口还要在校验用户态基础上，校验用户是否为管理员，以防止恶意请求使用普通用户账号操作管理员接口导致数据错误。

2.2.6 用户界面需求

用户界面设计尽量简洁，没有花哨的配色，图书借阅功能要简单化、一键化，在线阅读功能要尽可能减少对读者的干扰，没有广告弹窗等。

2.2.7 需求优先级分析

第一优先级：图书上传/下载功能、在线阅读功能；

第二优先级：用户自主注册与自主登录、图书信息查看、图书借阅功能；

第三优先级：图书管理、图书分类管理、收藏管理、借阅管理；

第四优先级：关键字搜索、分类搜索、自动还书；

第五优先级：校验接口用户态、管理员接口校验。

以上优先级确定是依据图书在线阅读功能、用户与借阅功能、后台管理功能、搜索与自动还书功能、数据安全功能的顺序进行排序的，基于这个顺序可以按部就班地对功能进行开发，还能确保不会出现先开发的功能依赖后实现功能的情况。依据优先级排序有利于功能地快速实现。

2.3 业务分析

2.3.1 数据流图

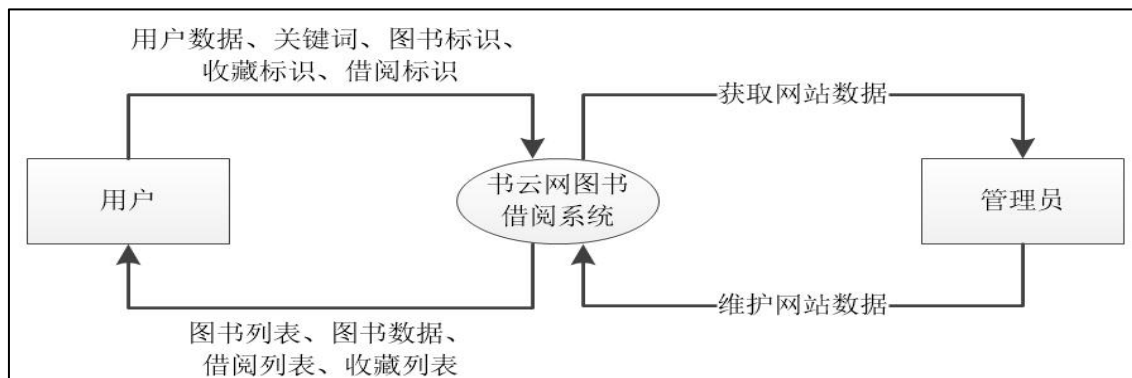


图 2-1 顶层数据流图

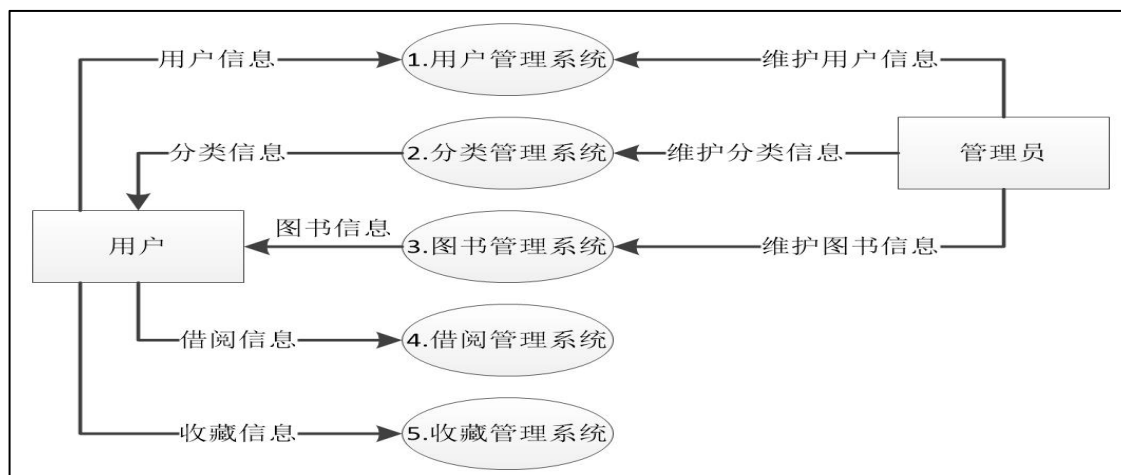


图 2-2 第 1 层数据流图

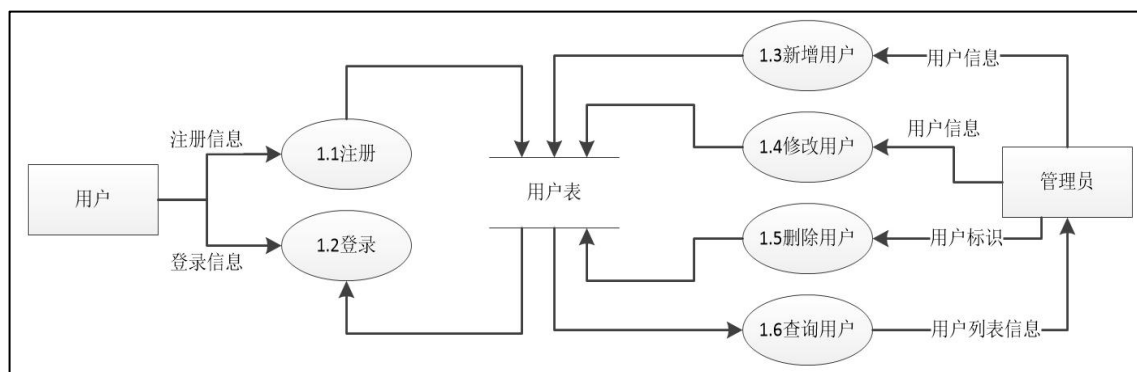


图 2-3 用户管理系统数据流图

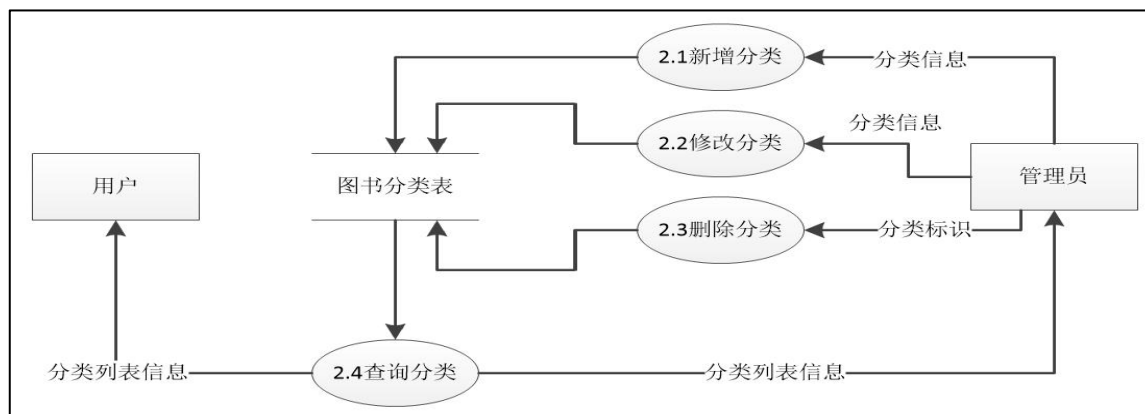


图 2-4 分类管理系统数据流图

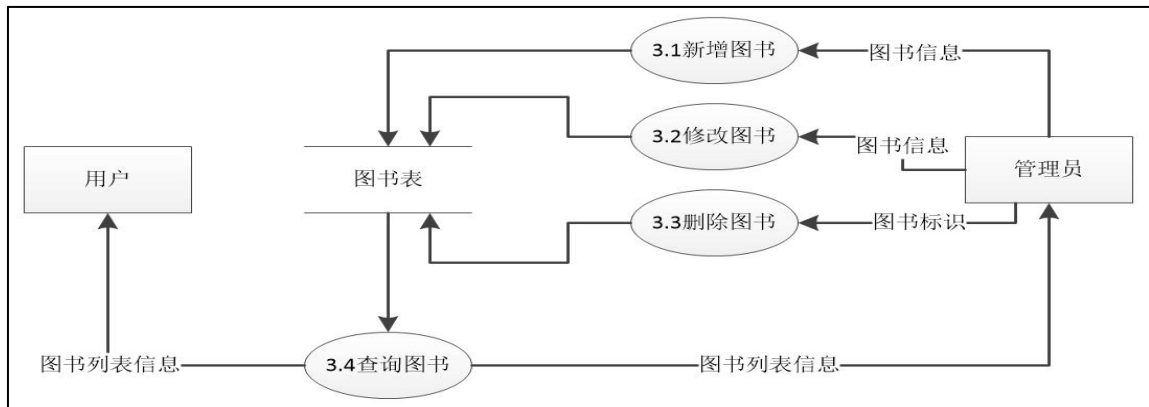


图 2-5 图书管理系统数据流图

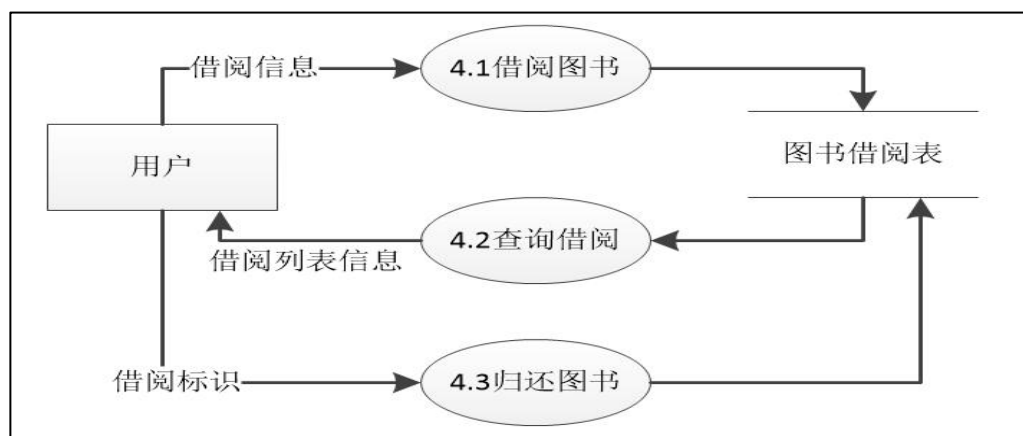


图 2-6 借阅管理系统数据流图

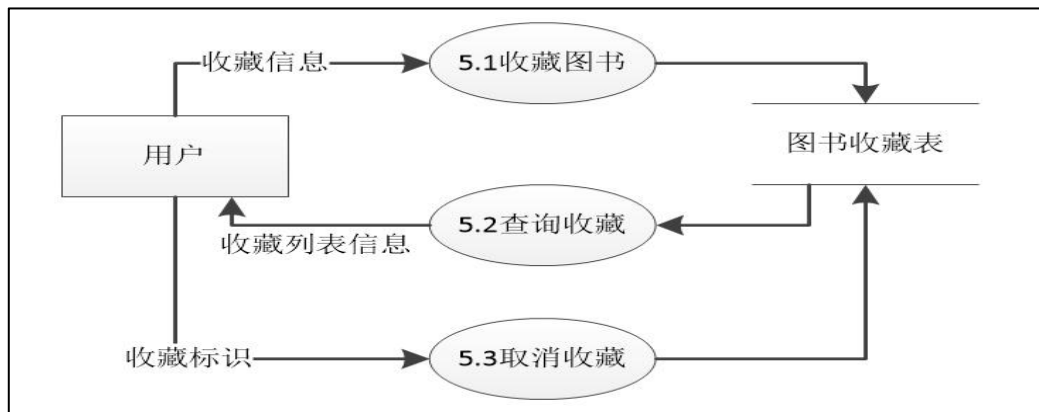


图 2-7 收藏管理系统数据流图

2.3.2 数据字典

数据字典由数据流与数据存储两种条目组成，以下分别列出。

数据流条目如下:

表 2-1 注册信息数据流条目

数据流名称	注册信息
说明	自行注册输入信息
数据流来源	用户
数据流去向	用户表
数据流组成	用户 id+用户名+用户密码

表 2-2 登录信息数据流条目

数据流名称	登录信息
说明	自行登录输入信息
数据流来源	用户
数据流去向	
数据流组成	用户名+用户密码

表 2-3 用户信息数据流条目

数据流名称	用户信息
说明	新增、修改用户时的信息
数据流来源	管理员
数据流去向	用户表
数据流组成	用户 id+用户名+用户密码+管理员标识

表 2-4 用户标识数据流条目

数据流名称	用户标识
说明	删除用户所使用的用户唯一标识
数据流来源	管理员
数据流去向	用户表
数据流组成	用户 id

表 2-5 用户列表信息数据流条目

数据流名称	用户列表信息
说明	查询用户时返回的用户列表信息
数据流来源	用户表
数据流去向	管理员
数据流组成	用户 id+用户名+管理员标识

表 2-6 分类信息数据流条目

数据流名称	分类信息
说明	新增、修改分类时的信息
数据流来源	管理员
数据流去向	图书分类表
数据流组成	分类 id+分类名称

表 2-7 分类标识数据流条目

数据流名称	分类标识
说明	删除分类的唯一标识
数据流来源	管理员
数据流去向	图书分类表
数据流组成	分类 id

表 2-8 分类列表信息数据流条目

数据流名称	分类列表信息
说明	查询分类列表返回的信息
数据流来源	图书分类表
数据流去向	用户、管理员
数据流组成	分类 id+分类名称

表 2-9 图书信息数据流条目

数据流名称	图书信息
说明	新增、修改图书时的信息
数据流来源	管理员
数据流去向	图书表
数据流组成	图书 id+名称+作者+出版社+出版日期+上架数+借出数

表 2-10 图书标识数据流条目

数据流名称	图书标识
说明	下架图书的唯一标识
数据流来源	管理员
数据流去向	图书表
数据流组成	图书 id

表 2-11 图书列表信息数据流条目

数据流名称	图书列表信息
说明	查询图书返回的图书列表信息
数据流来源	图书表
数据流去向	用户、管理员
数据流组成	图书 id+名称+作者+出版社+出版日期+上架数+借出数

表 2-12 借阅信息数据流条目

数据流名称	借阅信息
说明	借阅图书相关的信息
数据流来源	用户
数据流去向	图书借阅表
数据流组成	借阅 id+用户 id+图书 id+借阅时间+归还时间+状态标识

表 2-13 借阅列表信息数据流条目

数据流名称	借阅列表信息
说明	查询借阅记录时返回的借阅列表信息
数据流来源	图书借阅表
数据流去向	用户
数据流组成	借阅 id+用户 id+图书 id+借阅时间+归还时间+状态标识

表 2-14 借阅标识数据流条目

数据流名称	借阅标识
说明	提前归还图书时待操作的借阅记录唯一标识
数据流来源	用户
数据流去向	图书借阅表
数据流组成	借阅 id

表 2-15 收藏信息数据流条目

数据流名称	收藏信息
说明	添加收藏的信息
数据流来源	用户
数据流去向	图书收藏表
数据流组成	收藏 id+图书 id+用户 id+收藏时间

表 2-16 收藏列表信息数据流条目

数据流名称	收藏列表信息
说明	查询收藏时返回的列表数据
数据流来源	图书收藏表
数据流去向	用户
数据流组成	收藏 id+图书 id+用户 id+收藏时间

表 2-17 收藏标识数据流条目

数据流名称	收藏标识
说明	取消收藏使用的收藏唯一标识
数据流来源	用户
数据流去向	图书收藏表
数据流组成	收藏 id

数据存储条目如下：

表 2-18 用户表数据存储条目

数据存储名称	用户表
编号	D1
说明	存储用户信息
结构	用户 id+用户名+密码+管理员标识
有关数据流	注册信息、登录信息、用户信息、用户标识、用户列表信息

表 2-19 分类表数据存储条目

数据存储名称	分类表
编号	D2
说明	存储分类信息
结构	分类 id+分类名称
有关数据流	分类信息、分类标识、分类列表信息

表 2-20 图书表数据存储条目

数据存储名称	图书表
编号	D3
说明	存储图书信息
结构	图书 id+名称+作者+出版社+出版日期+上架数等
有关数据流	图书信息、图书标识、图书列表信息

表 2-21 图书借阅表数据存储条目

数据存储名称	图书借阅表
编号	D4
说明	存储图书借阅信息
结构	借阅 id+用户 id+图书 id+时间+归还时间
有关数据流	借阅信息、借阅标识、借阅列表信息

表 2-22 图书收藏表数据存储条目

数据存储名称	图书收藏表
编号	D5
说明	存储图书收藏信息
结构	收藏 id+用户 id+图书 id+收藏时间
有关数据流	收藏信息、收藏标识、收藏列表信息

2.4 可行性分析

2.4.1 经济可行性

一般新系统所需投资包括以下四个方面的内容，即：

- ① 设备费用：测试环境采用笔记本的虚拟机部署，几乎可不考虑设备费用。
- ② 人员费用：独立个人空闲时开发，不需要考虑人员费用。
- ③ 材料费用：采用全面电子化，不需要考虑材料费用。
- ④ 其他费用：无。

2.4.2 技术可行性

分析技术的可行性 3 个问题要考虑：在一段时间内需求功能可否做完、如何确保软件质量、如何保证开发效率。

首先，“在一段时间内需求功能可否做完”这一问题，由于项目整体复杂性可控，关键技术难点已识别并且可以克服，答案是可以的。

其次，“如何确保软件质量”这一问题，可以说功能实现上没有问题，可能细节上会有点小瑕疵，整体上是符合企业开发中的质量要求的。

最后，“如何保证开发效率”这一问题，于效率上是可以确保完成设计与实现的，由于不是纯粹的商业软件，开发效率也不是第一考量。

2.4.3 管理可行性

管理可行性是指软件系统开发的必要性^[2]。由于本系统的是毕业设计的一环，没有领导层干涉，只需要本人确认此系统的开发是有必要的，其可行性就是肯定的。

2.4.4 开发环境可行性

开发环境可行性是指领导是否知晓并愿意投入资源的意愿。由于本人独立设计开发本系统没有领导且开发环境也相对单纯，所以开发环境也是可行的。

2.4.5 法律可行性

法律可行性涉及到软件设计能否上线发布，在线图书借阅系统本身是公益性质的学习平台，法律不会对会其发布有限制。主要和法律相关的是上传的电子刊物，可能会因版权问题触犯版权法。可以通过只上架公版图书（不受版权法保护的图书）来规避版权法规的追责。另外也可以收到通知后立即对涉事图书进行下架处理。

2.4.6 使用可行性

在线图书借阅系统提供的功能界面很简洁，力图使用户略作尝试即可流畅使用。整体使用流程很清晰：用户注册/登录、搜索图书或分类搜索、打开有兴趣的图书查看并借阅或收藏、对已借阅的图书进行在线阅读。另外，在上线前需要进行一定测试以确保功能可用。

2.4.7 可行性分析结论

综合经济、技术、管理、开发环境、法律、使用等可行性分析，证明在线图书借阅系统在这几方面都是可行的，这个方案也是可行的。

第三章 概要设计

3.1 架构设计

3.1.1 软件架构设计

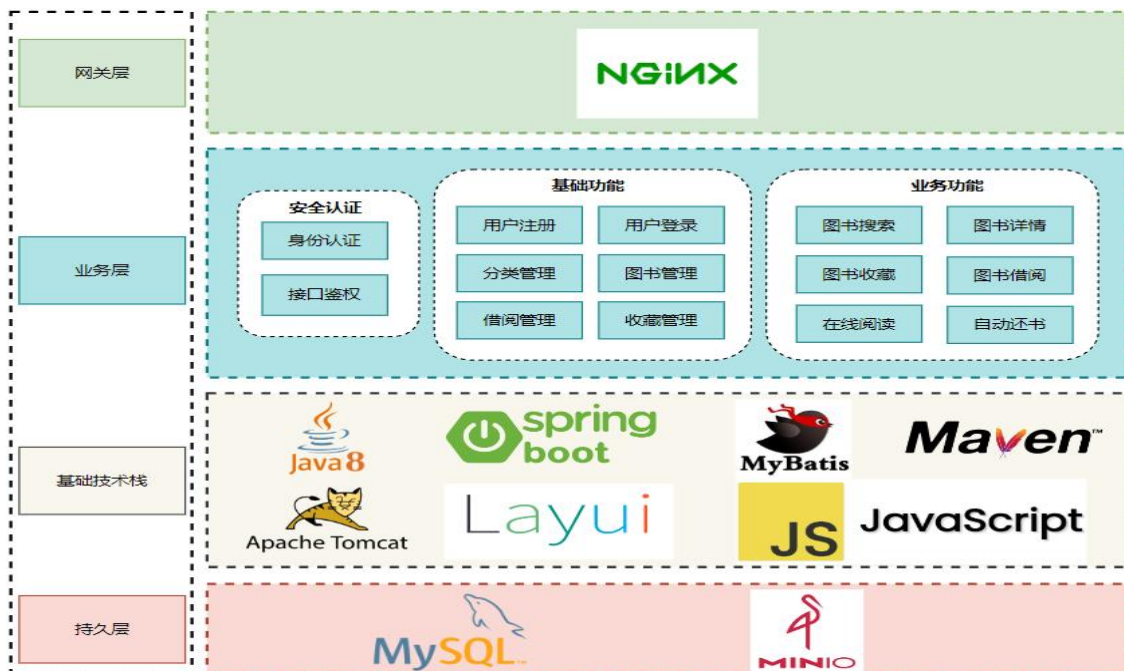


图 3-1 软件系统架构图

软件架构设计拆分为四个层次，由低到高分别为：持久层、基础技术栈、业务层、网关层。见图 3-2 所示。

持久层功能是对数据、二进制文件进行持久化存储，主要使用 MySQL 存储业务数据，使用 MinIO 存储 PDF 文件及图片文件。

基础技术栈体现在线图书借阅系统开发所使用到的技术栈，如后端开发使用 Java 语言 8.0 版本、后端框架使用 Spring Boot 以满足快速开发业务功能的目标、使用 MyBatis 将业务数据与 MySQL 交互存储、使用 Maven 对整个项目进行编译构建、使用内嵌 Tomcat 作为 Web 服务器、使用 Layui 组件库快速开发页面样式、使用 JavaScript 使界面功能更具动态化。

业务层包括三大块子功能模块，其中安全认证对系统进行访问控制和接口权限限制，提高了系统安全性。

基础功能是系统核心业务的基础数据来源，是整个业务系统的底座。

业务功能是系统核心业务，包含图书搜索、图书详情展示、图书收藏、图书借阅、在线阅读、自动还书 等功能，其中自动还书于用户是无感的。

网关层可使用 Nginx 提供系统的访问入口，性能较高的同时兼具 HTTPS、请求资源压缩、静态资源缓存、反向代理等高级功能，支持不停机添加或减少上游应用服务器，是理想的网关工具，在系统建设初期网关是可选的。

3.1.2 部署架构设计

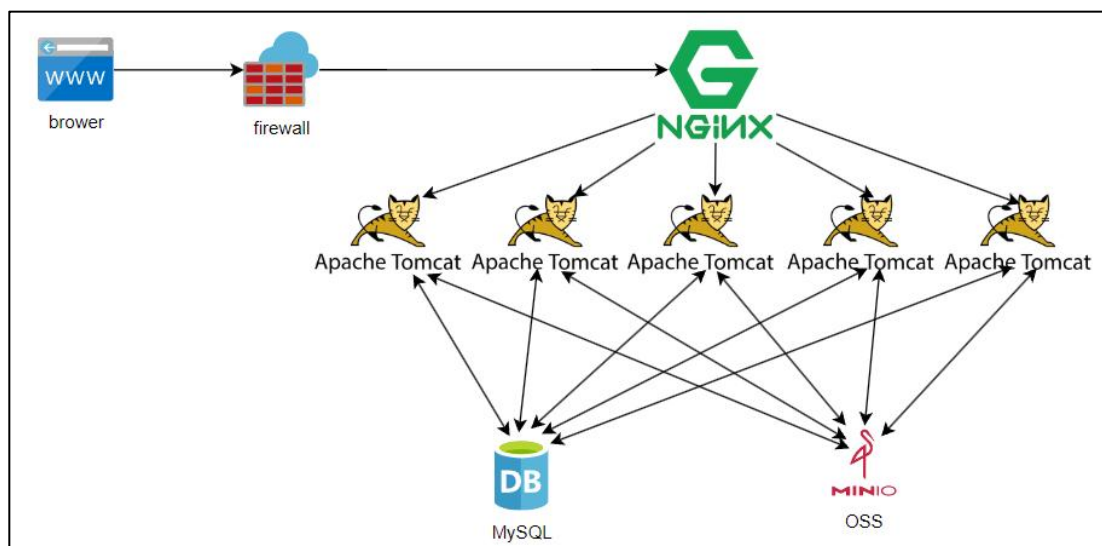


图 3-2 部署架构设计图

部署架构的设计能明确最终部署的模式。“一个网站的价值体现于能做到什么，而不关心如何做到。系统初建伊始就是小型网站，重点在于能为用户提供好的服务、创造用户价值并得到认可，实现可持续的业务发展^[3]”。受到这段话的启发，决定以业务为主，部署架构要尽量简单，如图 3-2 所示，用户通过浏览器输入项目的项目地址，经过网络路由到 Linux 服务器的 IP，通过 Linux 网络防火墙到达 Nginx 的 443 端口，Nginx 与浏览器建立 SSL 会话的同时会做好会话保持(支持集群扩容)，再将请求信息解密反向代理到 Tomcat（此时到达的流量是 HTTP 协议流量），Tomcat 根据请求的路径对静态资源（HTML/CSS/JavaScript）文件和动态数据接口进行区分，返回结果到 Nginx，再由 Nginx 将返回的资源交给浏览器渲染出页面，由 JavaScript 的脚本去获取动态数据完成动态数据渲染页面 DOM，最终呈现在用户面前。

之所以选择如图 3-2 的简单部署架构有两点重要考量：项目所处时期及用户量。项目目前所处初创时期，目标是根据软件架构和业务需求快速实现功能。在项目初期没有多少宣传的情况下，用户量不会太高。单节点 Nginx 就能实现支持 10 万个连接，单节点 Tomcat 最大支持 1 万个连接，何况还支持 Tomcat 做集群扩容，只要服务器资源足够大，最高同时支持 10 万个并发连接也是可能的，能充分满足系统初期阶段的性能需求。

3.2 数据库设计

3.2.1 数据库概念设计

数据库概念设计是指把业务模型抽象成概念模型，并使用 E-R 图来表示。如图 3-3，借阅关系是 M-N 的，也需要抽出一张表来设计。以下是系统总体 E-R 设计与 5 张业务表的数据库概念设计。

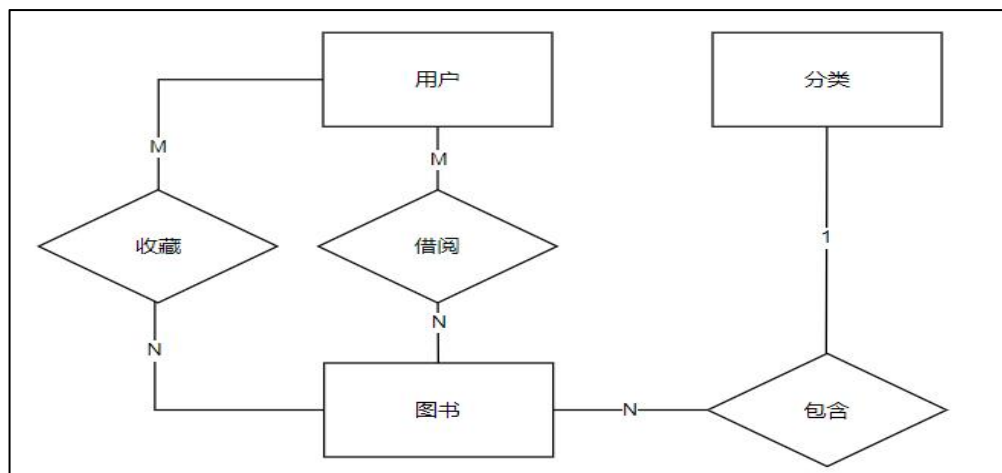


图 3-3 系统总体 E-R 图

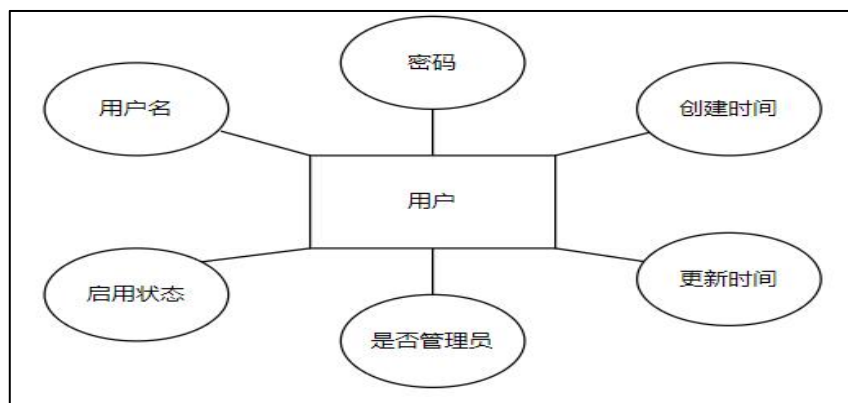


图 3-4 用户表 E-R 图

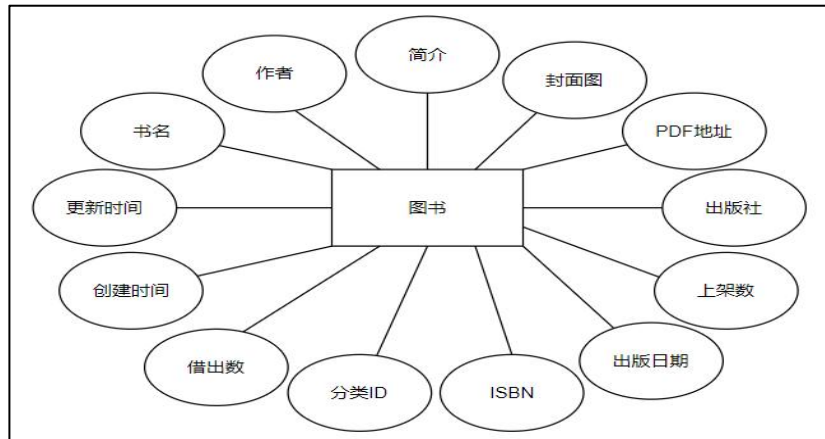


图 3-5 图书表 E-R 图

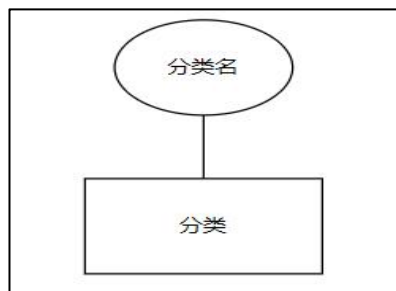


图 3-6 图书分类表 E-R 图

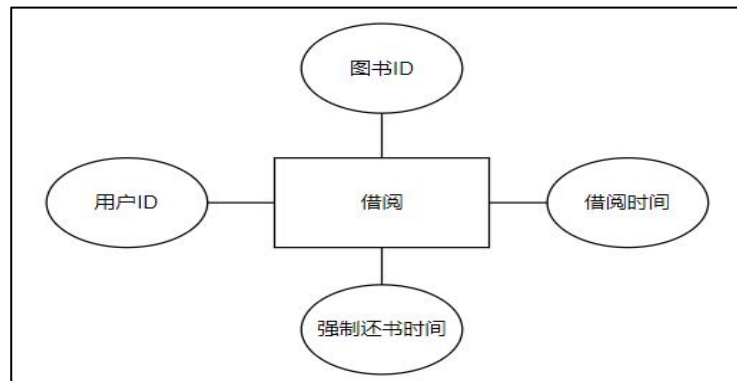


图 3-7 图书借阅表 E-R 图

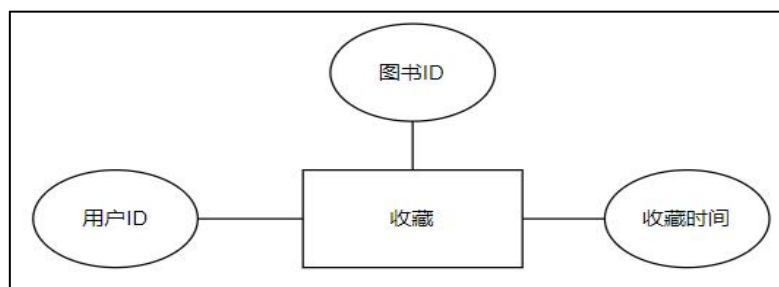


图 3-8 图书收藏表 E-R 图

3.2.2 数据库逻辑设计

数据库逻辑设计是将概念设计的 E-R 图转换为逻辑数据模型,可使用表格形式体现。

表 3-1 用户表(user)

字段名	数据类型	字段说明
ID	INT	主键 ID
USERNAME	VARCHAR(45)	用户名
PASSWORD	VARCHAR(256)	密码 MD5 值
CREATE_TIME	TIMESTAMP	创建时间
UPDATE_TIME	TIMESTAMP	更新时间
ENABLED	BOOLEAN	启用状态: false 禁用; true 启用
IS_ADMIN	BOOLEAN	是否管理员: false 不是; true 是

表 3-2 图书表(book)

字段名	数据类型	字段说明
ID	INT	主键 ID
NAME	VARCHAR(128)	图书名
AUTHOR	VARCHAR(45)	作者
INTRO	TEXT	图书简介
COVER	VARCHAR(256)	图书封面
PRESS	VARCHAR(256)	出版社名称
PUBLICATION_DATE	DATE	出版日期
ISBN	VARCHAR(17)	ISBN 编号
CATEGORY_ID	INT	图书分类 ID
PDF_URL	VARCHAR(256)	PDF 地址串
TOTAL_SHELVES	INT	上架图书数 (单位: 本)
BORROW_COUNT	INT	借出数 (单位: 本)
CREATE_TIME	TIMESTAMP	创建时间
UPDATE_TIME	TIMESTAMP	更新时间

表 3-3 图书分类表(category)

字段名	数据类型	字段说明
ID	INT	主键 ID
NAME	VARCHAR(64)	分类名

表 3-4 图书借阅表(borrow_record)

字段名	数据类型	字段说明
ID	INT	主键 ID
USER_ID	INT	用户 ID
BOOK_ID	INT	图书 ID
BORROW_TIME	TIMESTAMP	借阅时间
FORCE_RETURN_TIME	TIMESTAMP	强制还书时间
RETURN_TIME	TIMESTAMP	还书时间
STATUS	TINYINT	借阅状态: 0 待归还; 1 提前归还; 2 强制归还

表 3-5 图书收藏表(collection)

字段名	数据类型	字段说明
ID	INT	主键 ID
USER_ID	INT	用户 ID
BOOK_ID	INT	图书 ID
CREATE_TIME	TIMESTAMP	收藏时间

3.2.3 数据库物理设计

物理设计是指将逻辑模型具体转化为指定数据库管理系统的物理模型, 以下使用 MySQL 数据库的 SQL 建表语句表示。

```
create table if not exists user(
id int comment 'ID' auto_increment primary key,
username varchar(45) comment '用户名' unique key,
`password` varchar(256) comment '密码(加密)',
create_time timestamp comment '创建时间',
update_time timestamp comment '更新时间',
enabled boolean comment '状态, false禁用, true启用',
is_admin boolean comment '是否为管理员, true是, false否'
);
```

图 3-9 用户表建表 SQL 语句

```
create table if not exists category(
id int comment 'ID' auto_increment primary key,
name varchar(64) not null comment '分类名' unique key
);
```

图 3-10 分类表建表 SQL 语句

```

create table if not exists book(
id int comment 'ID' auto_increment primary key,
name varchar(128) comment '书名',
author varchar(45) comment '作者',
intro text comment '简介',
cover varchar(256) comment '图书封面',
press varchar(256) comment '出版社',
publication_date date comment '出版日期',
isbn varchar(17) comment 'ISBN编号',
category_id int comment '分类id',
pdf_url varchar(256) comment 'PDF地址',
total_shelves int DEFAULT 0 comment '上架数',
borrow_count int DEFAULT 0 comment '借出数',
create_time timestamp comment '上架时间',
update_time timestamp comment '更新时间'
);

```

图 3-11 图书表建表 SQL 语句

```

create table if not exists borrow_record(
id int comment 'ID' auto_increment primary key,
user_id int comment '用户id',
book_id int not null comment '图书id',
borrow_time timestamp comment '借书时间',
force_return_time timestamp comment '强制还书时间',
return_time timestamp comment '还书时间',
status tinyint(1) default 0 comment '状态, 0等待归还, 1提前归还, 2强制归还'
);

```

图 3-12 图书借阅表建表 SQL 语句

```

create table if not exists collection(
id int comment 'ID' auto_increment primary key,
user_id int comment '用户id',
book_id int not null comment '图书id',
create_time timestamp comment '收藏时间'
);

```

图 3-13 图书收藏表建表 SQL 语句

3.3 模块设计

本系统模块设计参考需求分析数据流图与数据库概念设计, 可划分为 5 大模块, 如图 3-14 所示。

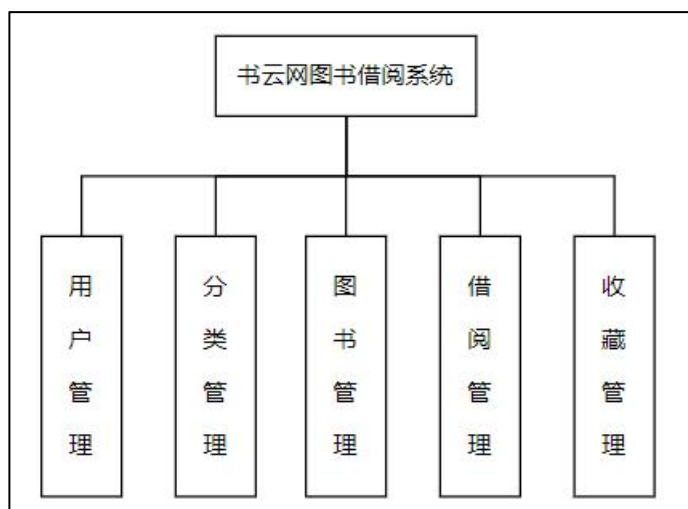


图 3-14 系统模块设计

3.4 系统功能设计

系统功能设计可理解为对模块设计结果拆解出各自模块应实现的功能，以下将对图 3-14 所示的模块分别介绍功能设计。

3.4.1 用户管理模块功能

对用户信息的管理是用户管理模块的职责，其功能包含用户自行注册与自行登录、查找用户、切换管理与启禁用状态、删除用户等 5 个功能，如图 3-15 所示。

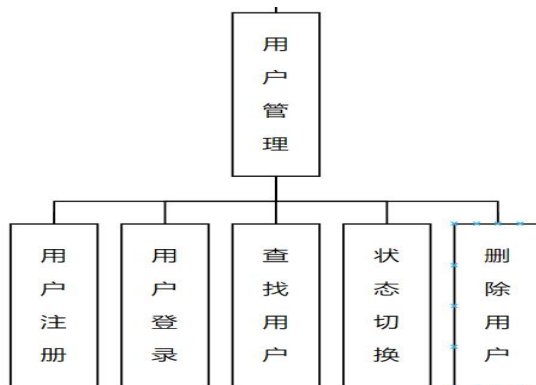


图 3-15 用户管理模块功能

3.4.2 分类管理模块功能

对图书分类信息的管理是分类管理模块的职责，其功能包含分类的新增、修改、查询与删除等 4 个功能，如图 3-16 所示。

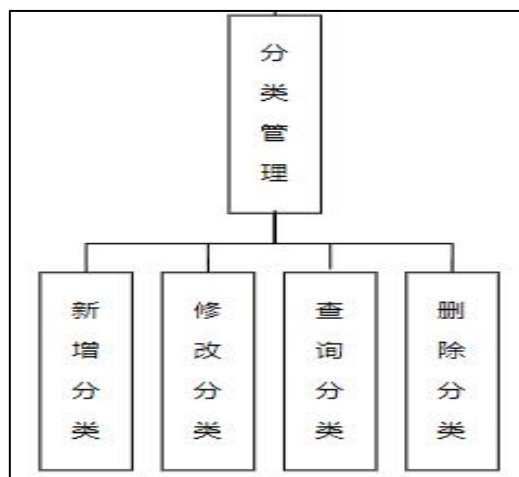


图 3-16 分类管理模块功能

3.4.3 图书管理模块功能

对图书信息的管理是图书管理模块的职责，其功能包含图书的上架、修改、搜索与下架等 4 个功能，如图 3-17 所示。

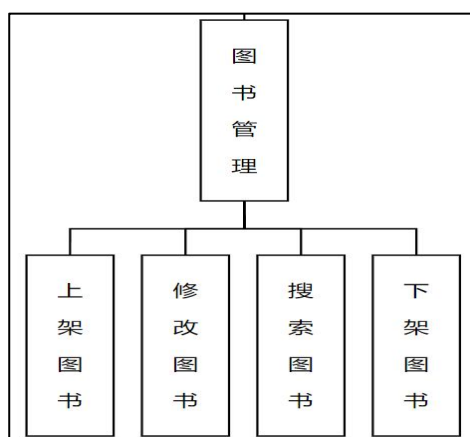


图 3-17 图书管理模块功能

3.4.4 借阅管理模块功能

对图书借阅信息的管理是借阅管理模块的职责，其功能包含图书借阅、在线阅读、提前归还、自动归还等 4 个功能，如图 3-18 所示。

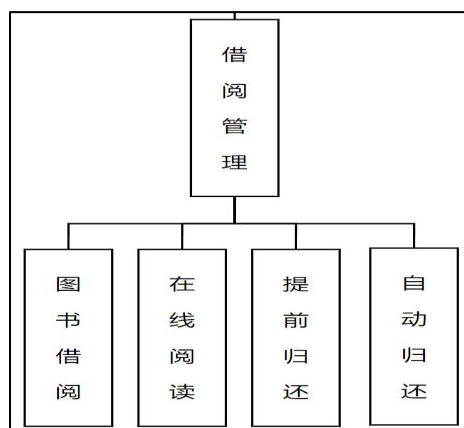


图 3-18 借阅管理模块功能

3.4.5 收藏管理模块功能

对图书收藏信息的管理是收藏管理模块职责，其功能包含图书收藏与取消收藏等 2 个功能，如图 3-19 所示。

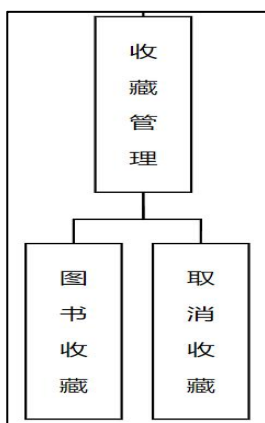


图 3-19 收藏管理模块功能

第四章 详细设计

4.1 界面设计

4.1.1 首页

首页的顶栏包含主/副标题和导航栏。其中主标题为“书云网”，副标题为“免费·限时借阅·云借书”。导航栏区分登录状态展示不同按钮，“首页”、“关于本站”、“登录”三个按钮是未登录态展示的，触发登录按钮则展示登录/注册模态框，登录成功后再根据管理员标识区分展示额外的按钮：普通用户显示“分类搜索”、“我的借阅”和“我

的收藏”这三个按钮；管理员用户则显示“分类管理”、“图书管理”和“用户管理”这三个按钮。

首页的主体居中显示图书搜索栏与搜索按钮，下方轮播展示一些名人名言的图片。其中搜索栏要响应回车键触发与按钮触发搜索功能跳转图书搜索结果页，用户未登录时弹框提示用户登录。

首页的底栏显示系统备案信息与开发者名称，在未正式上线前显示的均为静态信息。图 4-1 为首页的界面主体结构设计。

以上顶栏与底栏在除图书阅读页面不包含外，其余页面均采用此种方式展示相同的信息。

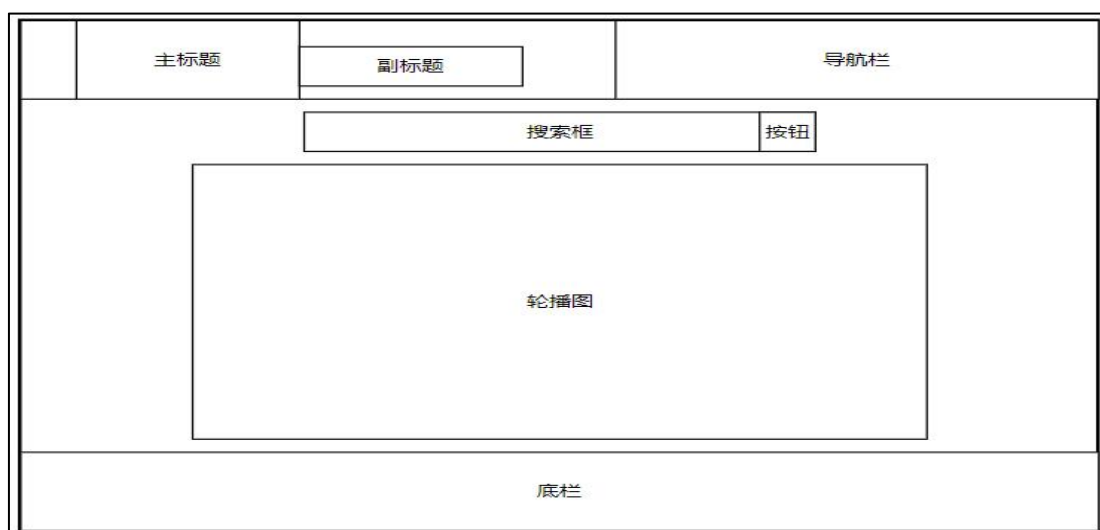


图 4-1 首页界面设计

4.1.2 用户注册页

注册页浮于界面之上，背景遮罩，由状态栏的“登录按钮”触发，由“切换注册”按钮展示注册页。业务逻辑是用户名、密码、确认密码输入框均不能为空，点击“注册”按钮校验输入参数合法性并与现存用户数据库表中现有用户进行比对，确认仅当无重名用户时才提示注册成功并两秒后自动跳转到登录页面。

该图展示了用户注册页面的布局。它包含以下元素：

- 标题：**“登录/注册”。
- 输入框：**“用户名输入框”、“密码输入框”和“确认密码输入框”。
- 按钮：**“切换登录界面按钮”和“注册”按钮。

图 4-2 用户注册页界面设计

4.1.3 用户登录页

登录页浮于界面之上，背景遮罩，由状态栏的“登录按钮”触发展示登录页。业务逻辑是触发登录按钮，对用户信息非空校验并查询比对用户信息，通过后才提示登录成功并关闭登录模态框，更新导航栏显示的按钮及用户名称。

登录/注册

用户名输入框

密码输入框

切换注册界面按钮

登录

图 4-3 用户登录页界面设计

4.1.4 图书搜索结果页

首页包含一处搜索功能，触发搜索按钮进入图书搜索结果页。搜索结果页展示查询结果列表，并支持图书的分类或关键字搜索、分页展示结果等功能。

	主标题	副标题	导航栏																												
分类竖排列表	关键字或分类: XXXX																														
	搜索框		按钮																												
	<table><thead><tr><th>ID</th><th>封面</th><th>书名</th><th>作者</th><th>ISBN</th><th>上架数</th><th>借出数</th></tr></thead><tbody><tr><td>1001</td><td> </td><td>XXXX</td><td>作者A</td><td></td><td>20</td><td>19</td></tr><tr><td>1002</td><td> </td><td>XXXX</td><td>作者B</td><td></td><td>20</td><td>5</td></tr><tr><td>1003</td><td> </td><td>XXXX</td><td>作者C</td><td></td><td>20</td><td>3</td></tr></tbody></table>				ID	封面	书名	作者	ISBN	上架数	借出数	1001		XXXX	作者A		20	19	1002		XXXX	作者B		20	5	1003		XXXX	作者C		20
ID	封面	书名	作者	ISBN	上架数	借出数																									
1001		XXXX	作者A		20	19																									
1002		XXXX	作者B		20	5																									
1003		XXXX	作者C		20	3																									
<< 1 2 3 4 5 6 >>																															
底栏																															

图 4-4 图书搜索结果页界面设计

4.1.5 图书详情页

图书详情页除了展示图书的信息外，还要展示用户对当前图书的收藏状态、借阅状态、在线阅读按钮。如未收藏展示为有底色的“收藏”按钮；如已收藏则展示为无底色的“取消收藏”按钮。如未借阅展示为有底色的“借阅”按钮，同时“在线阅读”按钮无底色，此时点击“在线阅读”弹框提示“请先借阅”字样；如已借阅则展示为无底色的“提前还书”按钮，同时“在线阅读”按钮有底色，点击跳转在线阅读页面。

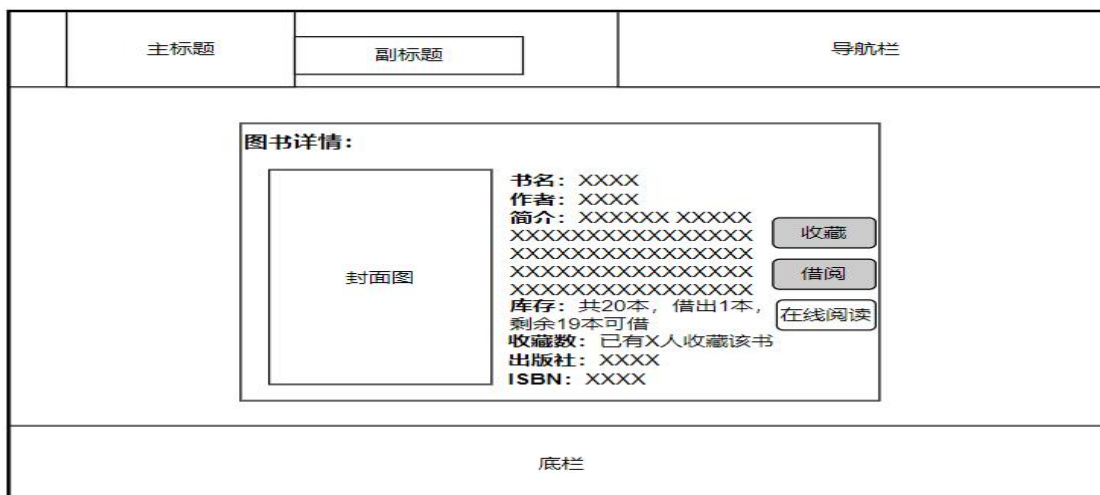


图 4-5 图书详情页界面设计

4.1.6 图书阅读页

图书阅读页尽量简单，把标题栏、导航栏、底栏都不作体现，以保持读者注意力集中到 PDF 电子书的阅读上。



图 4-6 图书阅读页界面设计

4.1.7 图书管理页

	主标题	副标题	导航栏																																												
图书管理 上架图书 刷新数据 搜索框 按钮 <table><thead><tr><th>ID</th><th>图书名称</th><th>作者</th><th>封面</th><th>出版社</th><th>出版时间</th><th>ISBN</th><th>分类名称</th><th>上架数</th><th>借出数</th><th>操 作</th></tr></thead><tbody><tr><td>1001</td><td>XXXX</td><td>作者A</td><td></td><td>A</td><td>xxxx</td><td>xxxx</td><td>计算机</td><td>20</td><td>19</td><td> 编辑 下架 </td></tr><tr><td>1002</td><td>XXXX</td><td>作者B</td><td></td><td>B</td><td>xxxx</td><td>xxxx</td><td>计算机</td><td>20</td><td>5</td><td> 编辑 下架 </td></tr><tr><td>1003</td><td>XXXX</td><td>作者C</td><td></td><td>C</td><td>xxxx</td><td>xxxx</td><td>计算机</td><td>20</td><td>3</td><td> 编辑 下架 </td></tr></tbody></table> << 1 2 3 4 5 6 >>				ID	图书名称	作者	封面	出版社	出版时间	ISBN	分类名称	上架数	借出数	操 作	1001	XXXX	作者A		A	xxxx	xxxx	计算机	20	19	编辑 下架	1002	XXXX	作者B		B	xxxx	xxxx	计算机	20	5	编辑 下架	1003	XXXX	作者C		C	xxxx	xxxx	计算机	20	3	编辑 下架
ID	图书名称	作者	封面	出版社	出版时间	ISBN	分类名称	上架数	借出数	操 作																																					
1001	XXXX	作者A		A	xxxx	xxxx	计算机	20	19	编辑 下架																																					
1002	XXXX	作者B		B	xxxx	xxxx	计算机	20	5	编辑 下架																																					
1003	XXXX	作者C		C	xxxx	xxxx	计算机	20	3	编辑 下架																																					
底栏																																															

图 4-7 图书管理页界面设计

4.1.8 图书分类管理页

	主标题	副标题	导航栏												
分类管理 添加分类 刷新数据 <table><thead><tr><th>分类ID</th><th>分类名称</th><th>操 作</th></tr></thead><tbody><tr><td>1001</td><td>计算机</td><td> 编辑 删除 </td></tr><tr><td>1002</td><td>计算机</td><td> 编辑 删除 </td></tr><tr><td>1003</td><td>计算机</td><td> 编辑 删除 </td></tr></tbody></table> << 1 2 3 4 5 6 >>				分类ID	分类名称	操 作	1001	计算机	编辑 删除	1002	计算机	编辑 删除	1003	计算机	编辑 删除
分类ID	分类名称	操 作													
1001	计算机	编辑 删除													
1002	计算机	编辑 删除													
1003	计算机	编辑 删除													
底栏															

图 4-8 图书分类管理页界面设计

4.1.9 借阅管理页

借阅管理又称为“我的借阅”，读者可管理已借阅的图书，立即阅读、查看图书详情、对未到期的借阅记录提前还书、对已到期的借阅记录再次借阅。

	主标题	副标题	导航栏							
我的借阅										
ID	图书名称	作者	借阅时间	强制还书时间	还书时间	状态	操 作			
1001	XXXXXXX	xxx	2024-02-14	2024-02-19		等待归还	立即阅读	详情	还书	再次借阅
1002	XXXXXXX	xxx	2024-02-12	2024-02-17	2024-02-17	强制归还	立即阅读	详情	还书	再次借阅
1003	XXXXXXX	xxx	2024-02-14	2024-02-19	2024-02-15	提前归还	立即阅读	详情	还书	再次借阅
1004	XXXXXXX	xxx	2024-02-14	2024-02-19		等待归还	立即阅读	详情	还书	再次借阅
1005	XXXXXXX	xxx	2024-02-12	2024-02-17	2024-02-17	强制归还	立即阅读	详情	还书	再次借阅
1006	XXXXXXX	xxx	2024-02-14	2024-02-19	2024-02-15	提前归还	立即阅读	详情	还书	再次借阅
1007	XXXXXXX	xxx	2024-02-12	2024-02-17	2024-02-17	强制归还	立即阅读	详情	还书	再次借阅
<<123456>>										
底栏										

图 4-9 借阅管理页界面设计

4.1.10 收藏管理页

收藏管理又称为“我的收藏”，读者可管理已收藏的图书信息，查看图书详情与取消收藏。

	主标题	副标题	导航栏																																																
我的收藏 <table><tr><th>ID</th><th>图书名称</th><th>作者</th><th>收藏时间</th><th colspan="2">操 作</th></tr><tr><td>1001</td><td>XXXXXX</td><td>xxx</td><td>2024-02-14 20:00:00</td><td>图书详情</td><td>取消收藏</td></tr><tr><td>1001</td><td>XXXXXX</td><td>xxx</td><td>2024-02-14 20:00:00</td><td>图书详情</td><td>取消收藏</td></tr><tr><td>1001</td><td>XXXXXX</td><td>xxx</td><td>2024-02-14 20:00:00</td><td>图书详情</td><td>取消收藏</td></tr><tr><td>1001</td><td>XXXXXX</td><td>xxx</td><td>2024-02-14 20:00:00</td><td>图书详情</td><td>取消收藏</td></tr><tr><td>1001</td><td>XXXXXX</td><td>xxx</td><td>2024-02-14 20:00:00</td><td>图书详情</td><td>取消收藏</td></tr><tr><td>1001</td><td>XXXXXX</td><td>xxx</td><td>2024-02-14 20:00:00</td><td>图书详情</td><td>取消收藏</td></tr><tr><td>1001</td><td>XXXXXX</td><td>xxx</td><td>2024-02-14 20:00:00</td><td>图书详情</td><td>取消收藏</td></tr></table> << 1 2 3 4 5 6 >>				ID	图书名称	作者	收藏时间	操 作		1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏	1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏	1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏	1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏	1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏	1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏	1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏
ID	图书名称	作者	收藏时间	操 作																																															
1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏																																														
1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏																																														
1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏																																														
1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏																																														
1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏																																														
1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏																																														
1001	XXXXXX	xxx	2024-02-14 20:00:00	图书详情	取消收藏																																														
底栏																																																			

图 4-10 收藏管理页界面设计

4.1.11 关于本站页

关于本站页面简要说明要 web 站点的设计人、平台建设目标及建议反馈方式。



图 4-11 关于本站页界面设计

4.2 逻辑设计

4.2.1 用户管理模块功能逻辑

用户注册功能逻辑设计，如图 4-12 所示。

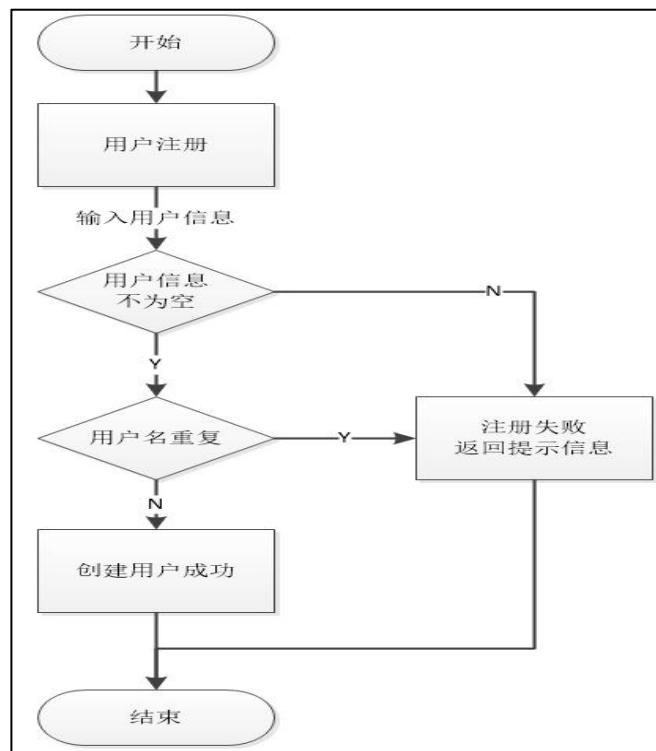


图 4-12 用户注册功能逻辑

用户登录功能逻辑设计，如图 4-13 所示。

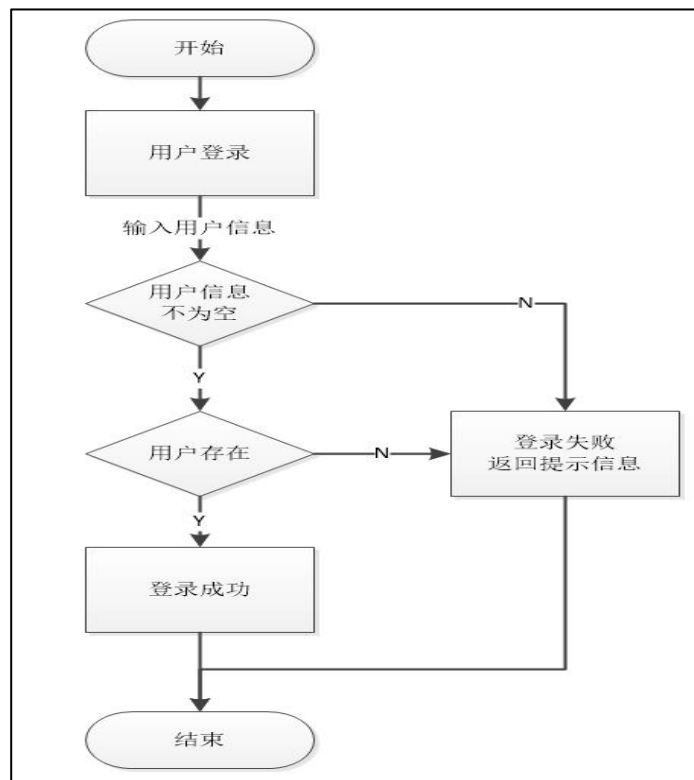


图 4-13 用户登录功能逻辑

查找用户功能逻辑设计，如图 4-14 所示。

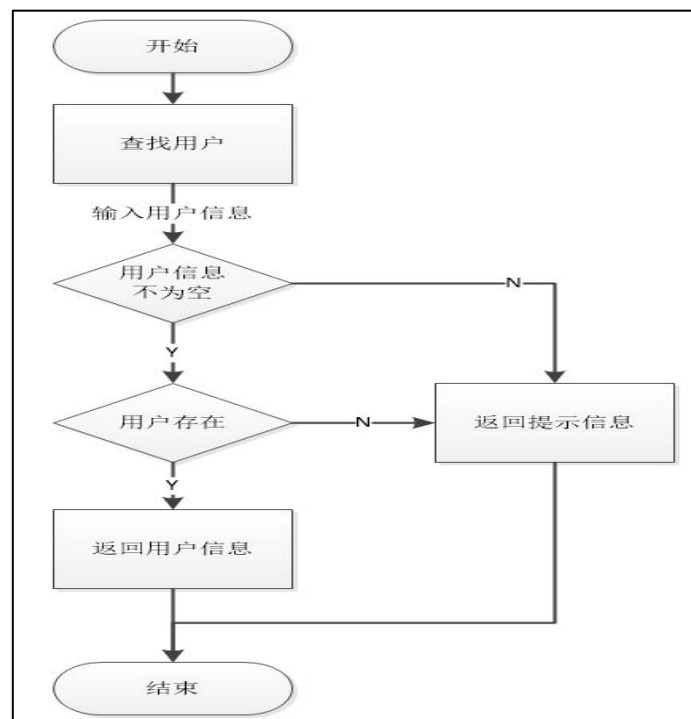


图 4-14 查找用户功能逻辑

管理员用户切换用户的管理员状态逻辑设计，如图 4-15 所示。

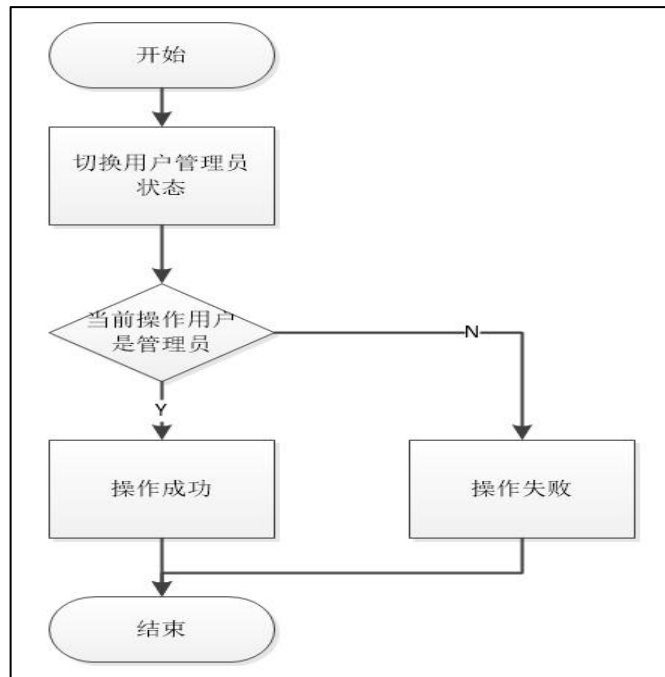


图 4-15 切换管理员状态功能逻辑

删除用户功能逻辑设计，如图 4-16 所示。

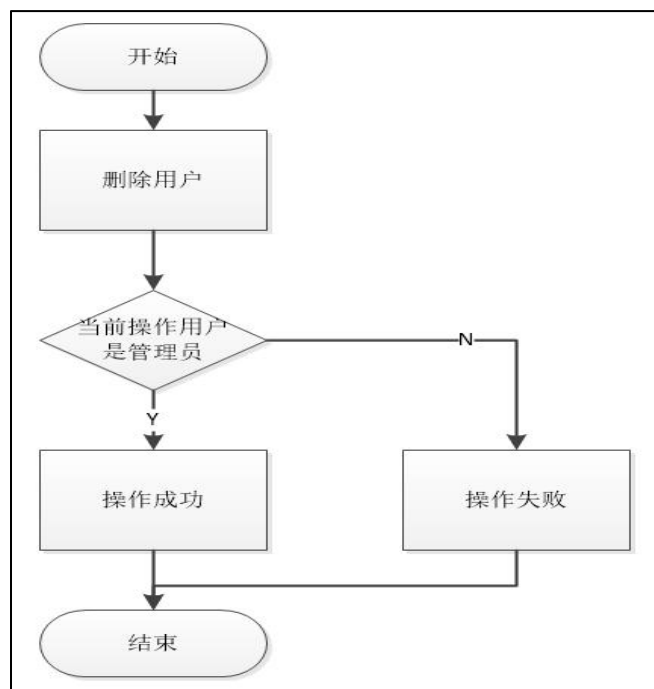


图 4-16 删除用户功能逻辑

4.2.2 分类管理模块功能逻辑

新增分类功能逻辑设计，如图 4-17 所示。

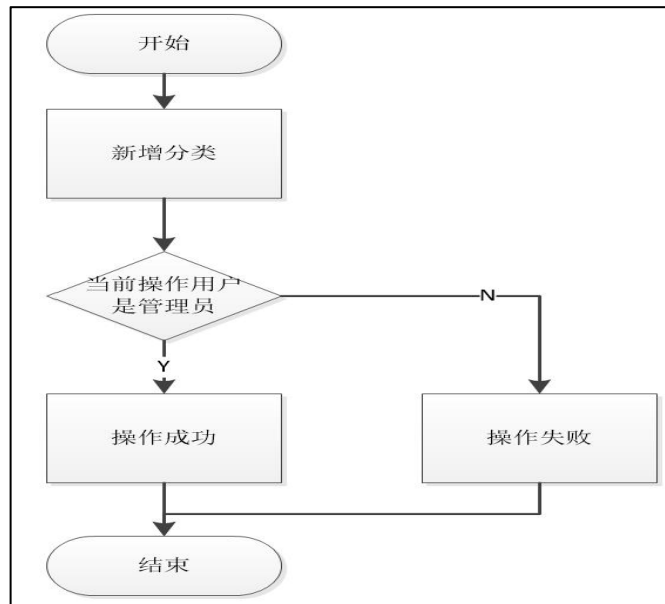


图 4-17 新增分类功能逻辑

修改分类功能逻辑设计，如图 4-18 所示。

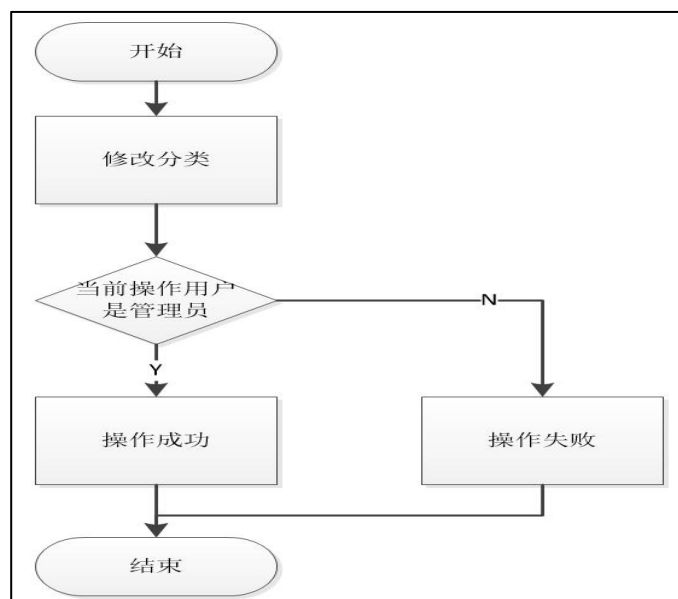


图 4-18 修改分类功能逻辑

查询分类功能逻辑设计，如图 4-19 所示。

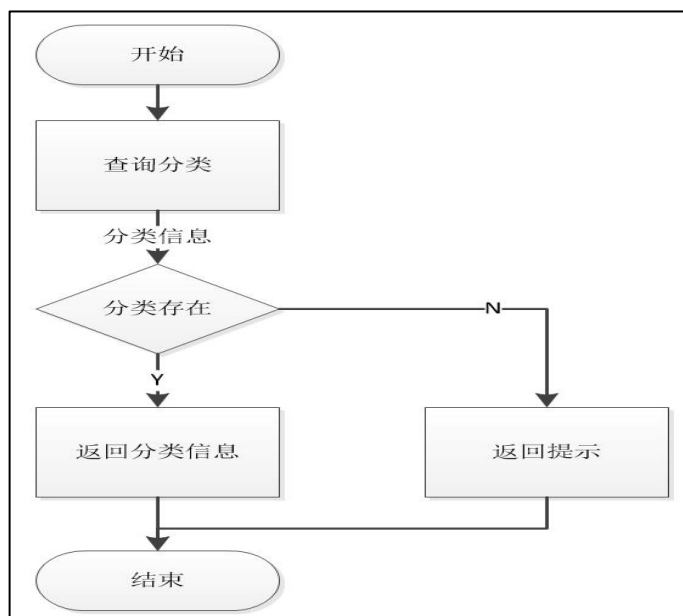


图 4-19 查询分类功能逻辑

删除分类功能逻辑设计，如图 4-20 所示。

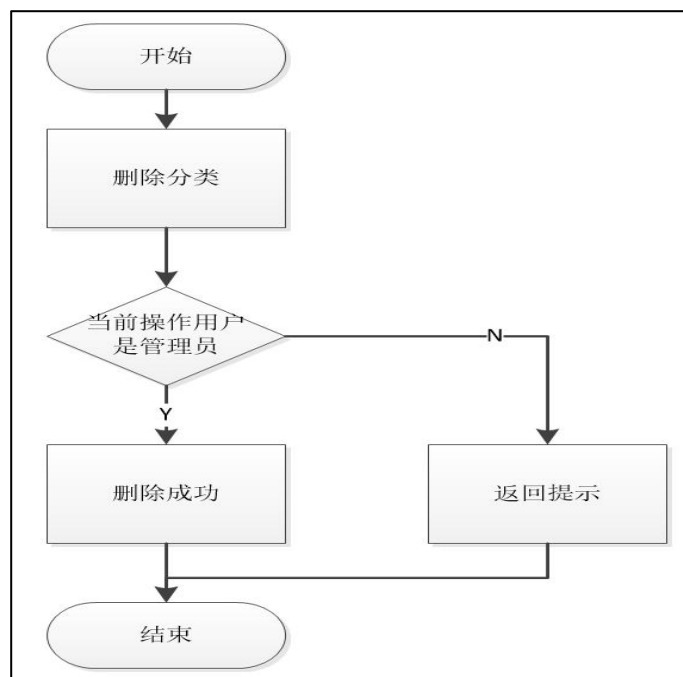


图 4-20 删除分类功能逻辑

4.2.3 图书管理模块功能逻辑

上架图书功能逻辑设计，如图 4-21 所示。

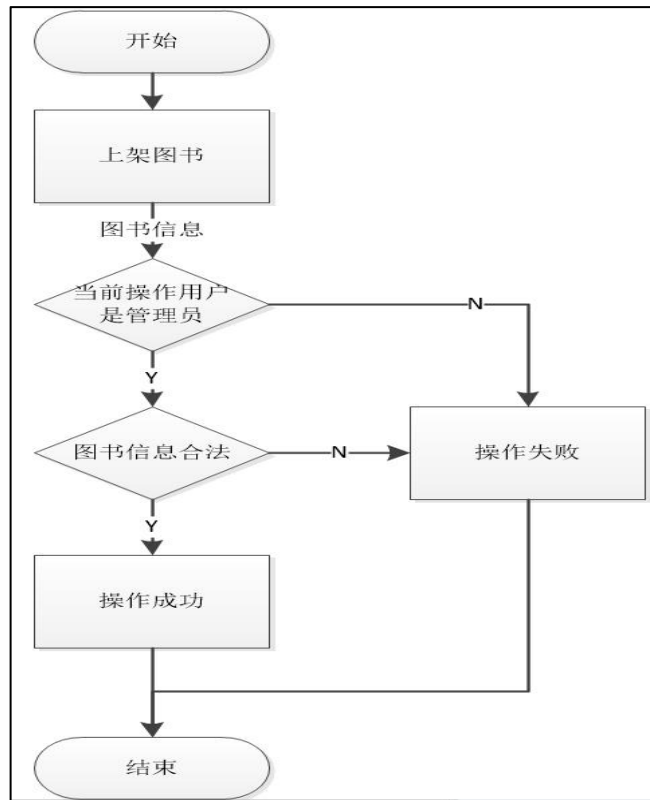


图 4-21 上架图书功能逻辑

修改图书功能逻辑设计，如图 4-22 所示。

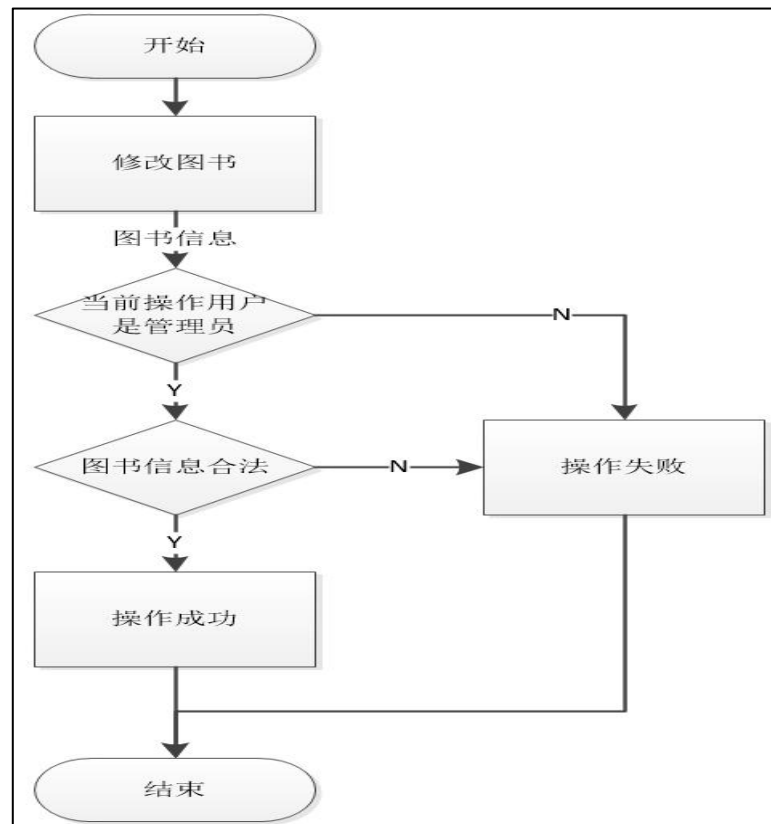


图 4-22 修改图书功能逻辑

搜索图书功能逻辑设计，如图 4-23 所示。

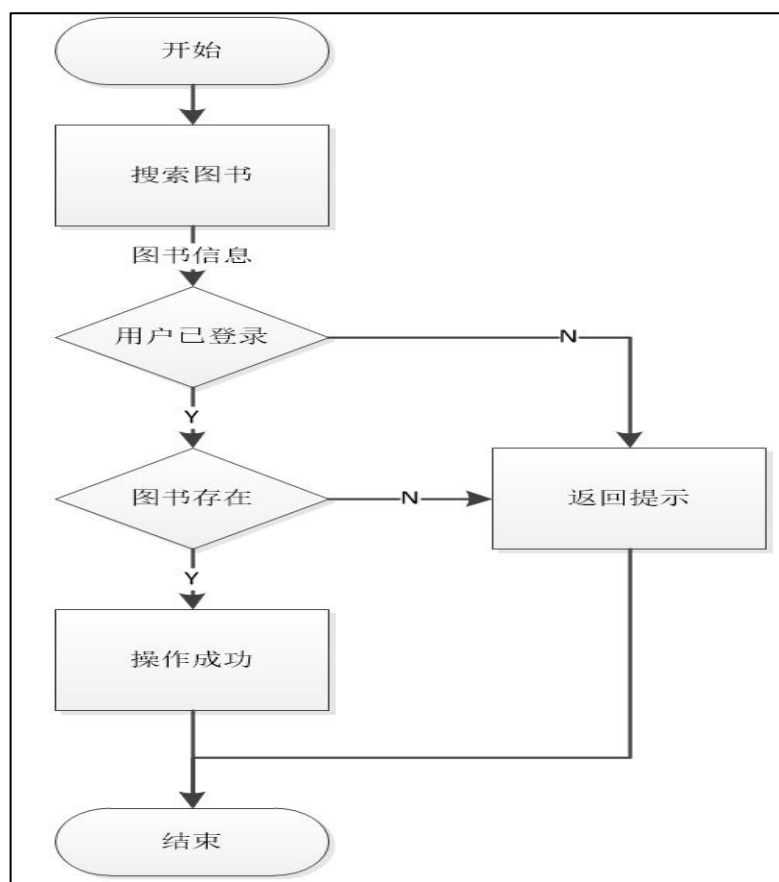


图 4-23 搜索图书功能逻辑

下架图书功能逻辑设计，如图 4-24 所示。

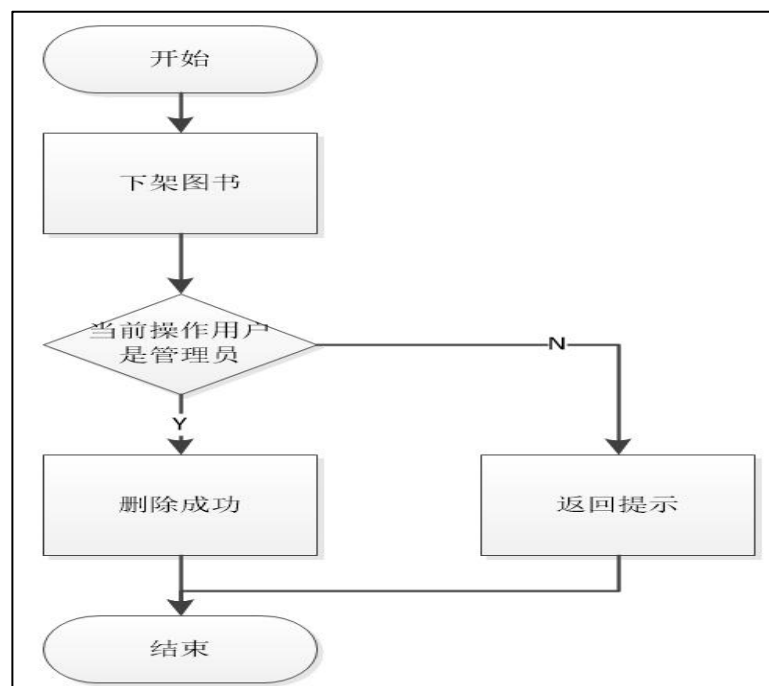


图 4-24 下架图书功能逻辑

4.2.4 借阅管理模块功能逻辑

图书借阅功能逻辑，如图 4-25 所示。

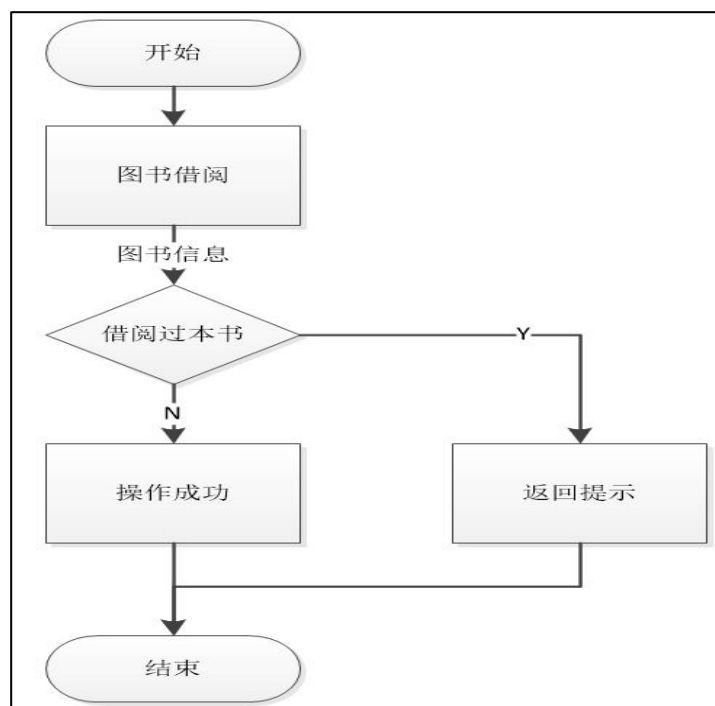


图 4-25 图书借阅功能逻辑

在线阅读功能逻辑，如图 4-26 所示。

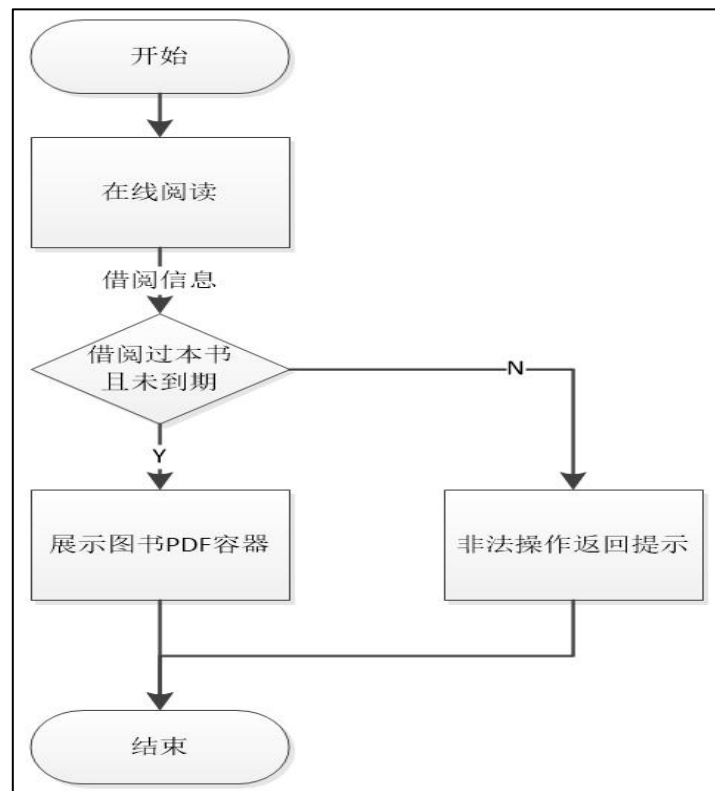


图 4-26 在线阅读功能逻辑

提前归还功能逻辑，如图 4-27 所示。

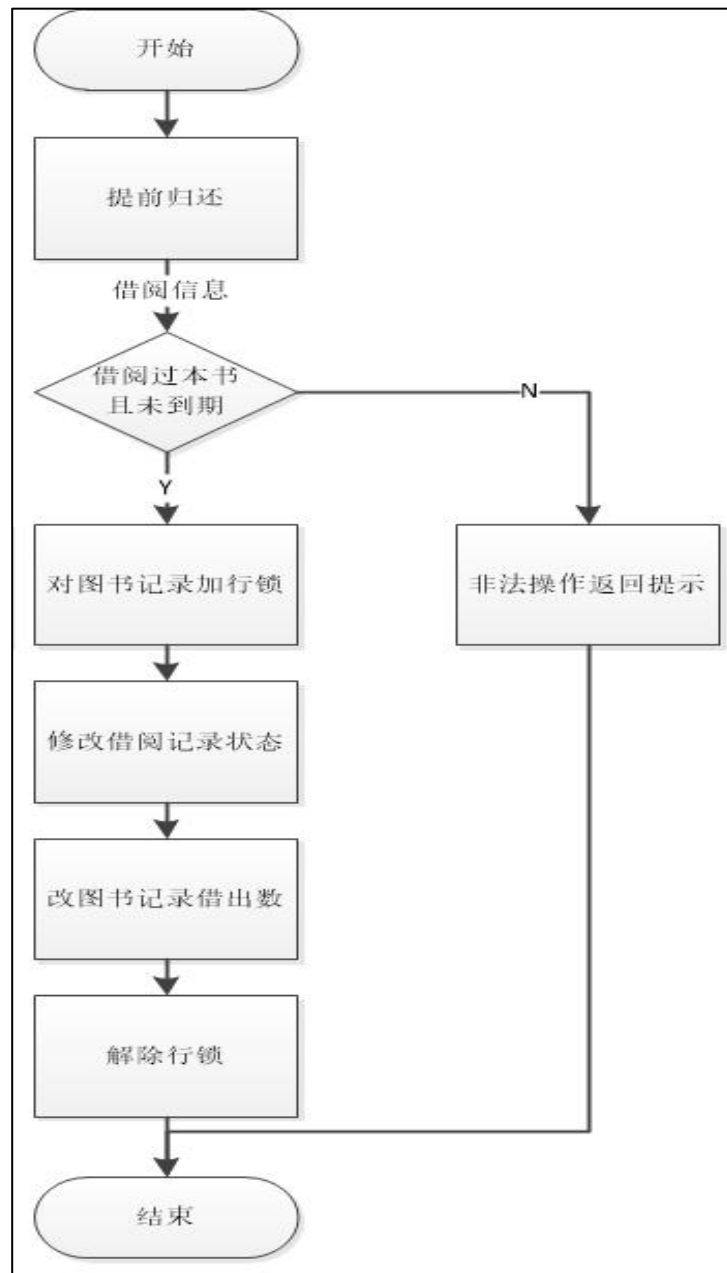


图 4-27 提前归还功能逻辑

自动归还功能逻辑，如图 4-28 所示。

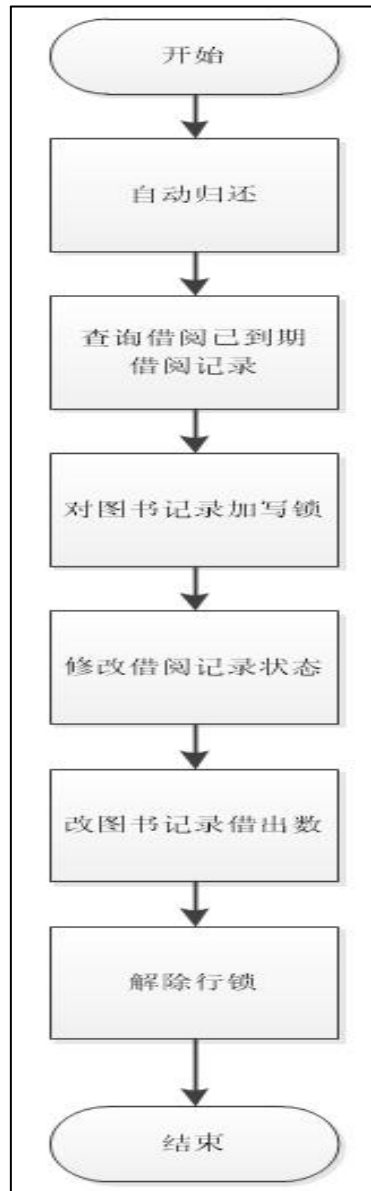


图 4-28 自动归还功能逻辑

4.2.5 收藏管理模块功能逻辑

图书收藏功能逻辑，如图 4-29 所示。

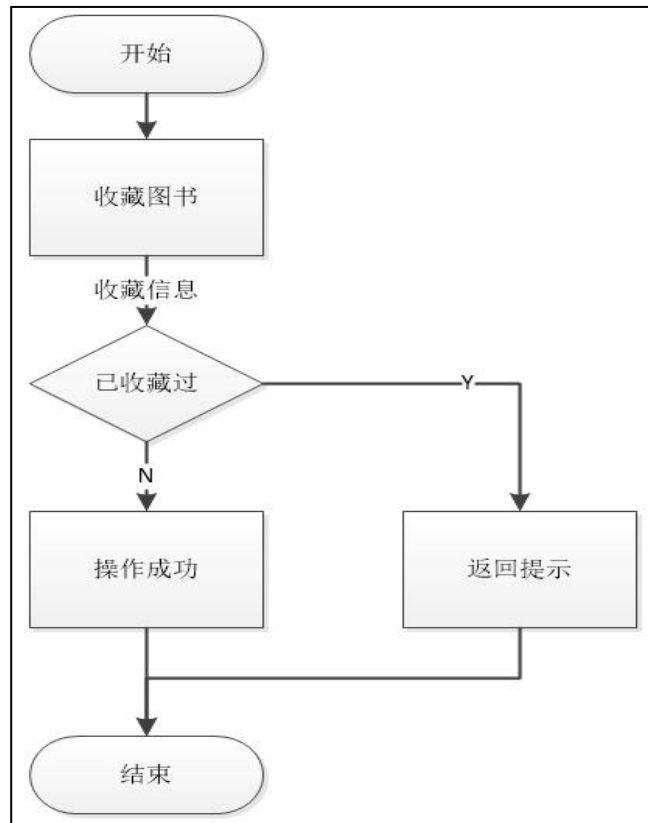


图 4-29 图书收藏功能逻辑

取消收藏功能逻辑，如图 4-30 所示。

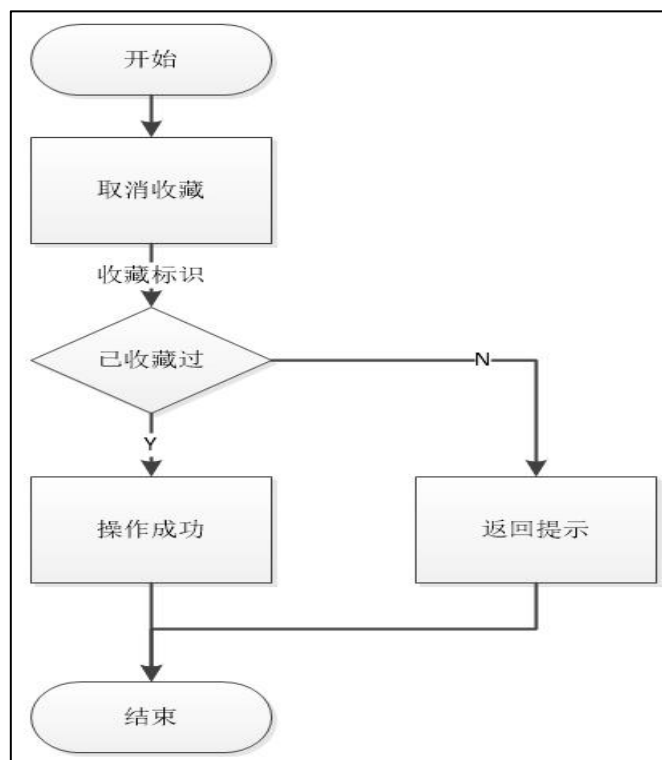


图 4-30 取消收藏功能逻辑

第五章 系统实现

5.1 开发语言的选择

“系统实现的第一步就是选择好程序设计语言及一些辅助开发工具^[4]”，编程语言的各种特性必然影响翻译到机器代码和网络通信过程的质量与效率。本系统实现中，采用 JavaScript 和 Java 这两种程序设计语言。

选择 JavaScript 语言开发前端主要原因是采用 B/S 架构，应用程序在用户侧由浏览器展示，JavaScript 语言可以直接运行于主流的浏览器，主流的浏览器对 JavaScript 的各种语言特性的支持都比较完备，网上的学习资源也较多，出现问题或 bug 在搜索引擎上都能搜索到解决方案，对开发人员比较友好。

选择 Java 语言开发后端的原因很多：一方面，作者在工作中主要是 Java 后端开发工程师，对 Java 语言相对熟练，实现功能效率较高，性能调优手段也较多；二方面，Java 语言不用关心内存的分配与释放，更容易写出健壮的代码；三方面，Java 虚拟机在编译成中间代码（class）时，会对代码进行一定优化（这点类似于 GCC 编译器），Java 虚拟机在运行过程中还会分析热点代码，对热点代码编译成本地代码来提升执行效率；四方面，“Write Once, Run Anywhere”是它的宣传语，天生跨平台能力强，得益于此它可在任意系统中运行。

5.2 软件开发工具的选择

软件开发工具是一系列工具的有机结合，可以帮助开发人员自动化、半自动化完成文档和代码的生成，以及自动以项目形式对代码和文档进行组织，熟练使用软件开发工具和其快捷按键，能极大提高开发人员的开发效率，减少重复性劳动。

选择合适的软件开发工具能提高开发效率，选择了不合适的开发工具可能会适得其反，那么应该如何选择软件开发工具，选择的标准又该有哪些呢？首先，也是最重要的——“软件开发工具一定要支持我们需要开发的语言”，不然工具说得再好都无济于事；其次，尽量选用开源免费的软件开发工具，这样能降低开发成本；最后，最好选用熟悉的软件开发工具，这样才能最大限度提高开发效率。由此可见，选择标准至少应包含：支持所需的语言、开源免费、尽可能选用熟悉的工具。前端与后端的软件开发工具可以是两套工具，毕竟术业有专攻，同一套软件开发工具可能侧重点不同，开发效率自然也不会相同。

前端开发工具选用 HBuilderX。它是专为前端开发而生的软件开发工具，具有丰富的功能和易于使用的特性。它整合了多种开发工具和语言，包括但不限于 HTML、CSS、

JavaScript 等。这些功能使得开发者能够更高效地进行 Web 前端开发。此外，它还具备轻巧、极速的特性和强大的语法提示功能，非常适合前端 Web 项目的开发。

后端开发工具选用 Eclipse。“Eclipse 是一个开源的 Java 语言开发的 IDE，可安装扩展 Plugins 以提高扩展性^[4]”。作为老牌的 Java 开发工具，以及目前本人工作中常用的 IDE，选择它再合适不过了。

5.3 模块关键代码

5.3.1 用户管理模块

用户注册对用户信息进行非空校验，查询用户在数据库中是否存在，如已存在则返回错误提示。

```
if (StringUtils.isBlank(userDTO.getUsername(), userDTO.getPassword())) {
    return R.error("用户名与密码不能为空");
}
User newUser = new User();
newUser.setUsername(userDTO.getUsername());
newUser.setPassword(DigestUtils.md5Hex(userDTO.getPassword()));
newUser.setAdmin(userDTO.getIsAdmin());
newUser.setEnabled(userDTO.getEnabled());
LocalDateTime now = LocalDateTime.now();
newUser.setCreateTime(now);
newUser.setUpdateTime(now);
return userService.add(newUser);
```

图 5-1 用户注册功能关键代码

用户登录按用户信息查询数据库中用户存在与否，并比对库中用户密文值与入参密码 MD5 值，比对一致则保存用户信息到 Session 中。创建 Session 时 Tomcat 会同时向浏览器发送 Cookie，其 key 为 JSESSIONID，值为 Session 的 ID。这样当登录过的用户的浏览器调用后端接口时，就能根据请求中携带的 Cookie 确定用户会话，从而实现用户登录状态保持功能。

```
User user = userMapper.findUserByUsername(userDTO.getUsername());
if (user != null && user.getEnabled()) {
    String md5Hex = DigestUtils.md5Hex(userDTO.getPassword());
    if (StringUtils.equals(user.getPassword(), md5Hex)) {
        HttpServletRequest request =
            ((ServletRequestAttributes) RequestContextHolder.getRequestAttributes())
                .getRequest();
        HttpSession session = request.getSession(false);
        if (session == null) {
            logger.info("用户({})会话不存在，创建会话并放置用户对象", user.getUsername());
            session = request.getSession();
            session.setAttribute("user", user);
        }
        return R.ok();
    } else {
        return R.error("用户名或密码输入错误");
    }
}
return R.error("该用户不存在或被禁用");
```

图 5-2 用户登录功能关键代码

查找用户功能使用用户 id 进行查找用户，另有支持查询用户列表功能。

```
public User findUserById(Integer userId) {
    return userMapper.findUserById(userId);
}
```

图 5-3 查找用户功能关键代码

管理员切换用户状态功能，包含切换启用禁用状态与管理员状态。

```
User addUser = userService.findUserById(userId);
if (addUser != null) {
    boolean needUpdate = false;
    if ("enabled".equals(option)) {
        addUser.setEnabled(!addUser.isEnabled());
        needUpdate = true;
    }
    if ("managed".equals(option)) {
        addUser.setAdmin(!addUser.isAdmin());
        needUpdate = true;
    }
    // 仅能更新非admin用户
    if (needUpdate && !"admin".equals(addUser.getUsername())) {
        userService.update(addUser);
    }
}
return R.ok();
```

图 5-4 切换用户管理员功能关键代码

删除用户功能按 userId 搜寻用户并删除已有用户，超管用户不允许删除。

```
User findUserById = userMapper.findUserById(userId);
// admin用户不允许删除
if (findUserById != null && !"admin".equals(findUserById.getUsername())) {
    userMapper.delete(userId);
}
```

图 5-5 删除用户功能关键代码

5.3.2 分类管理模块

新增分类功能校验参数不能带有分类 id。

```
if (category.getId() != null) {
    throw new BusinessException("保存时不能带上ID");
}
categoryMapper.save(category);
```

图 5-6 新增分类功能关键代码

修改分类功能校验参数必须有分类 id，以及分类存在才会更新。

```

if (category.getId() == null) {
    throw new BusinessException("更新时必须带上ID");
}
Category select = categoryMapper.select(category.getId());
if (select != null) {
    categoryMapper.update(category);
} else {
    throw new BusinessException("待更新分类不存在");
}

```

图 5-7 修改分类功能关键代码

查询分类功能使用分类 id 查询分类，另有支持查询分类列表的实现。

```

public Category findOne(Integer id) {
    return categoryMapper.select(id);
}

```

图 5-8 查询分类功能关键代码

删除分类功能参数前置校验管理员登录，再按图 5-9 执行删除分类。

```

Category select = categoryMapper.select(id);
if (select != null) {
    categoryMapper.delete(id);
}

```

图 5-9 删除分类功能关键代码

5.3.3 图书管理模块

上架图书功能先按图书名检查重复，再进行保存。

```

public void save(Book book) {
    Book exists = bookMapper.findBookByName(book.getName());
    if (exists != null) {
        throw new BusinessException("该书已添加，请勿重复上架");
    }
    bookMapper.save(book);
}

```

图 5-10 上架图书功能关键代码

修改图书功能先按图书 id 进行检查合法性，再对内容进行更新。

```

Book findBookById = bookMapper.findBookById(book.getId());
if (findBookById != null) {
    findBookById.setName(book.getName());
    findBookById.setAuthor(book.getAuthor());
    findBookById.setIntro(book.getIntro());
    findBookById.setCover(book.getCover());
    findBookById.setPress(book.getPress());
    findBookById.setPublicationDate(book.getPublicationDate());
    findBookById.setIsbn(book.getIsbn());
    findBookById.setCategoryId(book.getCategoryId());
    findBookById.setPdfUrl(book.getPdfUrl());
    findBookById.setTotalShelves(book.getTotalShelves());
    findBookById.setUpdateTime(LocalDateTime.now());
    bookMapper.update(findBookById);
    return;
}
throw new BusinessException("修改的图书未找到");

```

图 5-11 修改图书功能关键代码

搜索图书功能支持分类查询图书分页列表展示，另有按关键字搜索实现。

```
Page<List<Book>> page = new Page<>();
Integer count = bookMapper.findBookCountByCategoryId(categoryId);
Integer startPosition = Page.calculateStartPosition(currPage, pageSize);
List<Book> books = bookMapper.findBooksByCategoryId(categoryId, startPosition, pageSize);
page.setCount(count);
page.setCurr(currPage);
page.setLimit(pageSize);
page.setData(books);
return page;
```

图 5-12 搜索图书功能关键代码

下架图书功能前置校验管理员登录状，再校验图书 id 合法进行删除记录与对象。

```
Book findBookById = bookMapper.findBookById(bookId);
if (findBookById != null) {
    // 删除图片、pdf、图书记录
    deleteOssObject(findBookById.getPdfUrl());
    deleteOssObject(findBookById.getCover());
    bookMapper.delete(bookId);
}
```

图 5-13 下架图书功能关键代码

5.3.4 借阅管理模块

图书借阅功能先检验有无未还借阅记录，再使用 MySQL 行锁语法锁定所借图书，查询图书总上架数与借出数差值判断有无库存。有库存则更新图书借出数，添加借阅记录到借阅表中，方法执行完成数据库事务自动提交解除行锁。

```
Boolean recordExists = borrowRecordMapper.existsUnReturnBookBorrowRecord(bookId, userId);
if (recordExists) {
    throw new BusinessException("已有借阅记录，请勿重复借阅");
}
Book origin = bookMapper.findBookForUpdate(bookId);
int total = origin.getTotalShelves();
int borrowCount = origin.getBorrowCount();
if (total - borrowCount <= 0) {
    throw new BusinessException("已无库存");
}
int updateRow = bookMapper.updateBookBorrowCount(origin.getId(), borrowCount + 1);
if (updateRow > 0) {
    borrowRecordMapper.saveBorrowRecord(getBorrowRecord(bookId, userId));
} else {
    throw new BusinessException("扣库存失败");
}
```

图 5-14 图书借阅功能关键代码

在线阅读功能需确保用户登录状，查找图书信息与借阅过期时间返回给前端处理。

```

User userFromSession = SessionUtil.getUser();
if (userFromSession == null) {
    return R.error("阅读需登录");
}
Book book = bookService.findBookById(bookId);
String forceReturnTimeStr =
    borrowService.getUnReturnedRecordForceReturnTime(bookId, userFromSession.getId());
Map<String, Object> data = new HashMap<>();
data.put("forceReturnTimeStr", forceReturnTimeStr);
data.put("book", book);
return R.ok(data);

```

图 5-15 在线阅读功能关键代码

提前归还功能是用用户对未归还的图书进行还书，先查出未还借阅记录，再对图书信息加行锁，归还图书库存，再更新借阅记录为已还，方法执行结束数据库事务提交自动解除行锁。

```

Boolean recordExists = borrowRecordMapper.existsUnReturnBookBorrowRecord(bookId, userId);
if (!recordExists) {
    throw new BusinessException("没有借阅记录，非法操作");
}
List<BorrowRecord> unReturnBookBorrowRecords = borrowRecordMapper.selectUnReturnBookBorrowRecords(bookId,
    userId);
if (unReturnBookBorrowRecords == null || unReturnBookBorrowRecords.isEmpty()) {
    throw new BusinessException("不存在或存在多条同名图书借阅记录，数据不正确");
}
// 加锁读图书，还库存
Book findBookForUpdate = bookMapper.findBookForUpdate(bookId);
findBookForUpdate.setBorrowCount(findBookForUpdate.getBorrowCount() - 1);
bookMapper.update(findBookForUpdate);

// 更新借阅记录还书时间、状态
BorrowRecord borrowRecord = unReturnBookBorrowRecords.get(0);
borrowRecordMapper.updateBorrowRecordReturnState(borrowRecord.getId(), Consts.RETURNED, LocalDateTime.now());

```

图 5-16 提前归还功能关键代码

自动归还功能与提前归还类似，先查出所有过期未还的借阅记录，对图书信息进行加行锁，更新图书借出数再进行还书操作，即修改借阅记录状态。

```

List<BorrowRecord> borrowRecords = borrowRecordMapper.selectAllUnReturnOutDateBorrowRecords();
if (borrowRecords == null || borrowRecords.isEmpty()) {
    return 0;
}
Book book = null;
for (BorrowRecord borrowRecord : borrowRecords) {
    book = bookMapper.findBookById(borrowRecord.getBookId());
    if (book != null) {
        // 加行锁
        Book findBookForUpdate = bookMapper.findBookForUpdate(book.getId());
        findBookForUpdate.setBorrowCount(findBookForUpdate.getBorrowCount() - 1);
        bookMapper.update(findBookForUpdate);
        // 更新还书状态
        borrowRecordMapper.updateBorrowRecordReturnState(borrowRecord.getId(), Consts.RETURNED,
            LocalDateTime.now());
    }
}
return borrowRecords.size();

```

图 5-17 自动归还功能关键代码

5.3.5 收藏管理模块

图书收藏功能前置校验对收藏状态进行校验，在图 5-18 保存收藏记录。

```
public void collect(Integer bookId, Integer userId) {
    Collections c = new Collections();
    c.setUserId(userId);
    c.setBookId(bookId);
    c.setCreateTime(LocalDateTime.now());
    collectionMapper.save(c);
}
```

图 5-18 图书收藏功能关键代码

取消收藏功能先校验用户存在指定图书的收藏记录，对其删除操作。

```
Boolean exists = exists(bookId, userId);
if (exists) {
    Collections c = new Collections();
    c.setUserId(userId);
    c.setBookId(bookId);
    collectionMapper.delete(c);
}
```

图 5-19 取消收藏功能关键代码

5.4 系统部署运行

5.4.1 部署环境准备

准备一台装有 Ubuntu Linux 22.04 操作系统的服务器，禁用 swap 与 selinux，安装 openjdk8。以下为命令记录：

```
#禁用 swap
sudo swapoff -a
sudo sed -i "s#/swapfile#\#/swapfile#g" /etc/fstab
#禁用 selinux
setenforce 0
sudo sed -i "s#SELinux=enforcing#SELinux=disabled#g" /etc/sysconfig/selinux
#安装 openjdk8
sudo apt update
sudo apt install openjdk-8-jdk-headless
```

5.4.2 安装 MinIO 服务端

以下为命令记录：

```
#进入用户家目录
cd /home/hellxz
#下载 MinIO 二进制可执行文件并赋予可执行权限
wget https://dl.min.io/server/minio/release/linux-amd64/minio
```

```
chmod a+x minio
#创建 MinIO 数据存储目录
mkdir /home/hellxz/minio-data
#切换到 root 用户，创建 MinIO 服务，指定二进制文件及数据目录位置
#提供 MinIO 管理员用户名及密码
su root
cat > /etc/systemd/system/minio.service <<EOF
[Unit]
Description=Minio Service

[Service]
Environment="MINIO_ROOT_USER=admin"
Environment="MINIO_ROOT_PASSWORD=P@ssw0rd1994"
ExecStart=/home/hellxz/minio server /home/hellxz/minio-data --console-address
":9001"
ExecReload=/bin/kill -s HUP $MAINPID
ExecStop=/bin/kill -s QUIT $MAINPID
PrivateTmp=true

[Install]
WantedBy=multi-user.target
EOF
#重新加载服务，设置 MinIO 开机自启
systemctl daemon-reload
systemctl enable --now minio.service
#查看 MinIO 启动状态
systemctl status minio.service
```

5.4.3 安装 MySQL 数据库

安装 MySQL8.0 安装命令记录如下：

```
sudo apt update
sudo apt install mysql-server
sudo systemctl status mysql
#登录 MySQL，初始化 root 用户密码并授权限
mysql -u root
CREATE USER 'root'@'%' IDENTIFIED BY 'root';
```

```
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;  
FLUSH PRIVILEGES;
```

```
exit
```

#安全操作，关闭 root 用户远程登录，移除 test 库等

```
sudo mysql_secure_installation
```

使用数据库工具创建数据库 cloud-library，执行数据库初始化脚本。

5.4.4 编译项目源代码

使用 Maven 编译项目源代码，进入代码根目录，执行如下命令打包：

```
mvn clean package -DskipTests
```

程序 war 包生成在 target 目录下，得到 cloud-library-0.0.1.war。

5.4.5 启动项目

由于 Spring Boot 程序默认内嵌 Tomcat，所以可以直接通过命令启动程序。

```
nohup java -Dfile.encoding=UTF-8 -Djava.security.egd=file:/dev/./urandom -jar *.war  
&> app.log &
```

5.5 用户界面展示

用户界面实现较多，限于篇幅仅展示以下几个主要页面。



图 5-20 首页



图 5-21 搜索结果页



图 5-22 图书详情页



图 5-23 在线阅读页

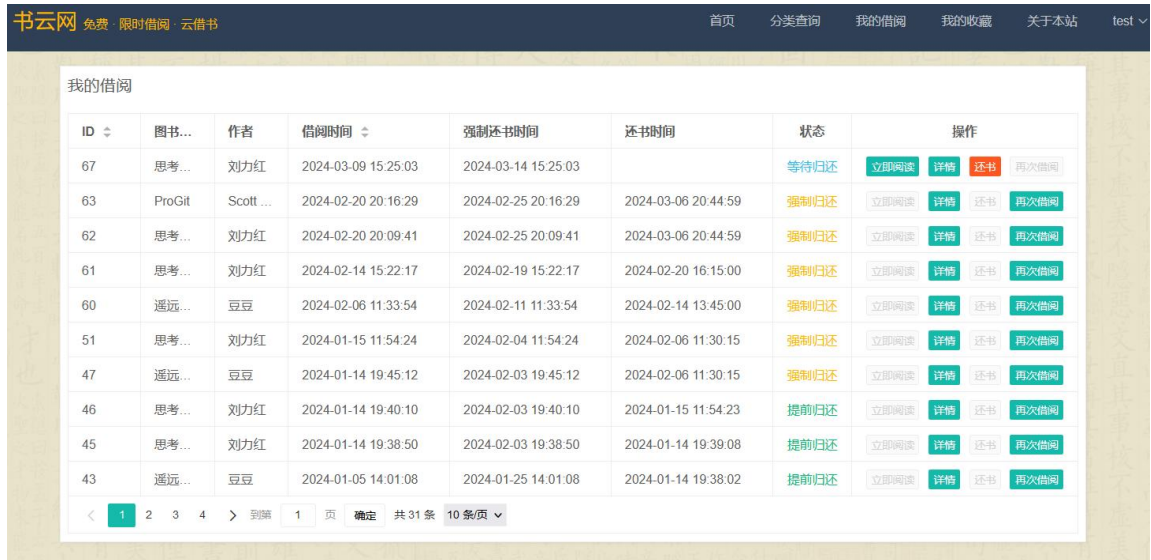


图 5-24 借阅管理页

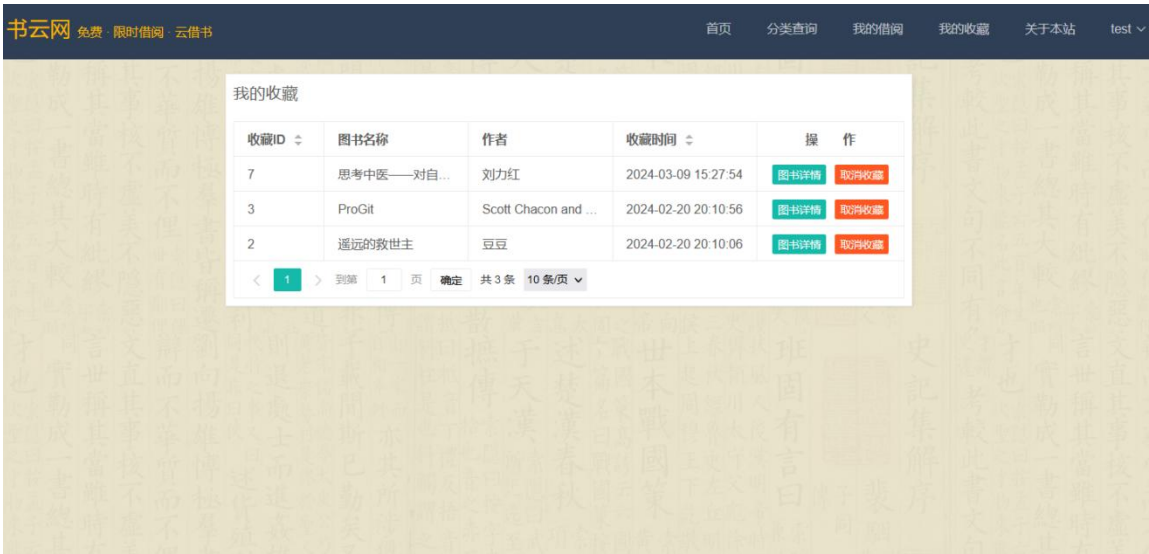


图 5-25 收藏管理页

第六章 系统测试

6.1 测试的目的与内容

“错误是不可避免的，因此发现错误是保障软件与产品质量的基础^[5]”。测试的重点是尽早发现错误并进行修改完善，是确定设计的功能是否能符合预期，如前端页面显示是否正常，后端是否报错等。

测试的内容是依据功能逻辑设计的流程图，按分支编写对应的测试用例并执行，对测试用例的执行结果进行分析与改进。

6.2 功能测试与结果

测试用例对逻辑设计中的流程图进行分支覆盖，以确保测试充分性。流程分支参见逻辑设计各功能流程图，在本节中不做过多描述。

6.2.1 用户管理模块功能测试

表 6-1 用户注册功能测试用例

测试用例	预期	结果
用户名、密码、确认密码均不输	错误提示	错误提示
只输入用户名，不输入密码、确认密码	错误提示	错误提示
输入用户名、密码，不输入确认密码	错误提示	错误提示

输入用户名、密码、确认密码	注册成功	注册成功
输入已有用户名，密码与确认密码不为空	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-2 用户登录功能测试用例

测试用例	预期	结果
用户名、密码均不输	错误提示	错误提示
输入用户名，不输入密码	错误提示	错误提示
输入密码，不输入用户名	错误提示	错误提示
输入存在用户名与正确密码	登录成功	登录成功
输入存在用户名与错误密码	错误提示	错误提示
输入不存在用户名与任意密码	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-3 查找用户功能测试用例

测试用例	预期	结果
管理员输入不存在用户名	错误提示	错误提示
管理员输入存在用户名	操作成功	操作成功
普通用户通过接口调用查找用户	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-4 切换用户状态功能测试用例

测试用例	预期	结果
管理员切换用户管理员标识状态为管理员	切换成功	切换成功
管理员切换用户管理员标识状态为非管理员	切换成功	切换成功
普通用户通过接口调用切换用户状态	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-5 删除用户功能测试用例

测试用例	预期	结果
管理员删除用户	操作成功	操作成功
普通用户通过接口调用删除用户	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

6.2.2 分类管理模块功能测试

表 6-6 新增分类功能测试用例

测试用例	预期	结果
管理员不输入分类名	错误提示	错误提示
管理员输入分类名	操作成功	操作成功

管理员输入已存在分类名	错误提示	错误提示
普通用户通过接口调用新增分类	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-7 修改分类功能测试用例

测试用例	预期	结果
管理员不输入分类名	错误提示	错误提示
管理员输入分类名	操作成功	操作成功
管理员输入已存在分类名	错误提示	错误提示
普通用户通过接口调用修改分类	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-8 查询分类功能测试用例

测试用例	预期	结果
管理员登录查找分类列表	操作成功	操作成功
普通用户登录查找分类列表	操作成功	操作成功
不登录查找分类列表	操作成功	操作成功

功能结果分析：分类查询无需登录，执行测试用例均符合预期，功能正确。

表 6-9 删除分类功能测试用例

测试用例	预期	结果
管理员删除分类	操作成功	操作成功
普通用户通过接口调用删除分类	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

6.2.3 图书管理模块功能测试

表 6-10 上架图书功能测试用例

测试用例	预期	结果
管理员不输入必填项，未上传图书	错误提示	错误提示
管理员输入必填项，未上传图书	错误提示	错误提示
管理员输入必填项，上传图书	操作成功	操作成功
普通用户通过接口调用上架图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-11 修改图书功能测试用例

测试用例	预期	结果
管理员删除回填信息中必填信息	错误提示	错误提示
管理员修改任意图书信息	操作成功	操作成功
普通用户通过接口调用修改图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-12 搜索图书功能测试用例

测试用例	预期	结果
管理员通过图书管理搜索图书	操作成功	操作成功
管理员通过首页搜索图书	操作成功	操作成功
不登录搜索图书	错误提示	错误提示
普通用户登录通过首页搜索图书	操作成功	操作成功

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-13 下架图书功能测试用例

测试用例	预期	结果
管理员下架图书	操作成功	操作成功
普通用户通过接口调用下架图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

6.2.4 借阅管理模块功能测试

表 6-14 图书借阅功能测试用例

测试用例	预期	结果
普通用户借阅未借过的图书	操作成功	操作成功
普通用户借阅已借过的图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-15 在线阅读功能测试用例

测试用例	预期	结果
普通用户查看已借过未过期的图书	操作成功	操作成功
普通用户查看已借过已过期的图书	错误提示	错误提示
普通用户接口调用查看未借过的图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-16 提前归还功能测试用例

测试用例	预期	结果
普通用户提前归还已借未过期图书	操作成功	操作成功
普通用户接口调用提前归还过期借阅的图书	错误提示	错误提示
普通用户接口调用提前归还未借状态图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-17 自动归还功能测试用例

测试用例	预期	结果
借阅图书后 5 天内查看借阅状态	未过期	未过期
借阅图书后 5 天后查看借阅状态	已过期	已过期

功能结果分析：执行测试用例均符合预期，功能正确。

6.2.5 收藏管理模块功能测试

表 6-18 图书收藏功能测试用例

测试用例	预期	结果
普通用户收藏未收藏的图书	操作成功	操作成功
普通用户调用接口收藏已收藏图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

表 6-19 取消收藏功能测试用例

测试用例	预期	结果
普通用户取消已收藏图书	操作成功	操作成功
普通用户调用接口取消未收藏图书	错误提示	错误提示

功能结果分析：执行测试用例均符合预期，功能正确。

第七章 总结

通过本次在线图书借阅系统的设计与实现,将自考本科阶段的计算机专业相关知识进行了一次梳理,理论联系实践,将《软件工程》课本中的需求分析、业务流程分析和结构化思想方法在分析设计工作中进行了充分的实践,将《信息系统开发与管理》课本中的结构化开发方法与《软件工程》中的软件测试在系统实现工作中也进行了一定的实践,使用《软件开发工具》中提到 Eclipse 开发工具开发系统后端代码,并依据开发工具的特点选择了更合适前端开发的 HbuilderX 工具。

除了应用课本中习得的知识和方法论外,独立参与本系统的设计与实现,也是对个人能力的一大提升,加深了对软件工程的理解与实际软件设计与开发的完整流程的认知,真正做到了理论联系实践与知行合一。也明晓了软件工程的方法论不仅仅能设计本系统这样相对简单的项目,更适合大型、超大型的复杂项目的设计、开发与管理。

最后,本系统整体使用流程比较顺利、学习曲线平滑,用户界面简洁大方,没有干扰阅读的广告和弹窗信息,基本满足了广大读者对在线借阅系统的期许,也满足了在线阅读方面和提高阅读学习效率方面的需求。

参考文献

- [1] 韩建芳,游凤霞.基于媒介技术哲学观的在线阅读微探[J].理论观察,2022(09):150-153
- [2] 刘世峰.信息系统开发与管理[M].北京:机械工业出版社,2011.3
- [3] 李智慧.大型网站技术架构:核心原理与案例分析[M].北京:电子工业出版社,2013.9
- [4] 陈禹,方美琪.软件开发工具[M].北京:机械工业出版社,2011.4
- [5] 王立福.软件工程[M].北京:机械工业出版社,2011.4

致 谢

感谢自考这个平台，没有它就没有通过学习提升学历的机会。感谢自考办的导师与工作人员的工作，没有你们开直播悉心讲解毕业设计论文的考核点和要素，本人很难写出良好的分析与设计。感谢初稿审核老师详细地标注出论文初稿中的不足与建议，没有您的建议本人不可能进行精确地调整。感谢家人在本人毕设与论文期间的关怀与理解，没有你们分担家庭琐事就没有我安心地去做设计开发。

附 录

附录 1 程序源代码清单:

— pom.xml	Maven 配置文件
— src	源代码目录
— main	
— java	后端代码目录
— online	
— hellxz	
— cloudlibrary	
— CloudLibraryApplication.java	项目启动类
— config	配置类目录
— domain	数据库对象目录
— dto	传输对象目录
— interceptor	拦截器目录
— mapper	MyBatis Mapper 接口目录
— misc	杂项目录
— runner	执行器目录
— service	业务接口目录
— impl	业务实现目录
— web	控制器目录
— resources	
— application.yml	SpringBoot 配置文件
— logback.xml	日志配置文件
— mapper	MyBatis 配置文件目录
— static	前端静态文件目录
— css	CSS 目录
— html	HTML 页面目录
— image	图片目录
— index.html	首页
— js	JS 目录
— layui	Layui 框架目录

代码文件共 107 个, 限于篇幅原因, 仅展示相对关键的 Java 代码与 JavaScript 代码。

```

@Configuration
public class WebMvcConfig implements WebMvcConfigurer {
    // 解决跨域问题
    @Override
    public void addCorsMappings(CorsRegistry registry) {
        // @formatter:off
        registry.addMapping("/**")
            .allowedOrigins("*")
            .allowedMethods("GET", "POST", "DELETE", "PUT", "OPTIONS")
            .allowCredentials(false)
            .allowedHeaders("*")
            .maxAge(3600);
        // @formatter:on
    }

    // 拦截器配置
    @Override
    public void addInterceptors(InterceptorRegistry registry) {
        // 注册用户登录拦截器
        registry.addInterceptor(new UserLoginInterceptor()).addPathPatterns("/**").excludePathPatterns("/",
            "/user/login", "/user/register", "/user/current", "/index.html", "/html/*", "/css/*", "/font/*",
            "/image/*", "/js/*", "/layui/*", "/favicon.ico");
        // 注册管理员权限拦截器
        registry.addInterceptor(new AdminAuthInterceptor()).addPathPatterns("/user/listPage", "/user/add",
            "/user/toggle", "/user/delete", "/book/listPage", "/book/add", "/book/edit", "/book/delete",
            "/category/listPage", "/category/add", "/category/delete", "/category/edit", "/upload");
    }
}

```

图附录-1 用户登录与管理接口拦截器配置

```

/**
 * 用户登录拦截器
 *
 * @author ZhangChengguang
 * @date 2024年2月1日
 */
public class UserLoginInterceptor implements HandlerInterceptor {

    /**
     * 请求到达控制器前置处理, 如果用户未登录, 写出提示信息, 禁止调用接口
     */
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler)
        throws Exception {
        User userFromSession = SessionUtil.getUser();
        if (userFromSession != null) {
            return true;
        } else {
            response.setCharacterEncoding("UTF-8");
            response.setContentType("text/html;charset=utf8");
            try(PrintWriter writer = response.getWriter()){
                writer.write("用户未登录, 请访问<a href=\"/\">首页</a>登录");
            }
            return false;
        }
    }
}

```

图附录-2 用户登录拦截器

```

/**
 * 管理权限控制拦截器
 *
 * @author ZhangChengguang
 * @date 2024年2月1日
 */
public class AdminAuthInterceptor implements HandlerInterceptor {

    /**
     * 请求到达控制器前置处理, 如果用户非管理员, 写出提示信息, 禁止调用接口
     */
    @Override
    public boolean preHandle(HttpServletRequest request, HttpServletResponse response, Object handler)
        throws Exception {
        User user = SessionUtil.getUser();
        if (user == null || !user.isAdmin()) {
            response.setCharacterEncoding("UTF-8");
            response.setContentType("text/html;charset=utf8");
            try(PrintWriter writer = response.getWriter()){
                writer.write("本接口仅管理员可调用");
            }
            return false;
        }
        return true;
    }
}

```

图附录-3 管理接口拦截器

```

public R login(UserDTO userDTO) {
    // 查出用户比密码加密值, 检查用户会话是否已登录过, 如登录过使用已有会话, 否则创建
    User user = userMapper.findUserByUsername(userDTO.getUsername());
    if (user != null && user.getEnabled()) {
        String md5Hex = DigestUtils.md5Hex(userDTO.getPassword());
        if (StringUtils.equals(user.getPassword(), md5Hex)) {
            HttpServletRequest request =
                ((ServletRequestAttributes) RequestContextHolder.getRequestAttributes())
                    .getRequest();
            HttpSession session = request.getSession(false);
            if (session == null) {
                logger.info("用户({})会话不存在, 创建会话并放置用户对象", user.getUsername());
                session = request.getSession();
                session.setAttribute("user", user);
            }
            return R.ok();
        } else {
            return R.error("用户名或密码输入错误");
        }
    }
    return R.error("该用户不存在或被禁用");
}

```

图附录-4 用户登录 Service 实现方法

```

public R add(User user) {
    Boolean hasUser = userMapper.exists(user.getUsername());
    if (!hasUser) {
        try {
            userMapper.save(user);
        } catch (Exception e) {
            logger.error("添加用户失败:" + e.getMessage(), e);
            return R.error("添加用户失败:" + e.getMessage());
        }
    } else {
        return R.error("该用户已存在, 请更换用户名再试");
    }
    return R.ok();
}

```

图附录-5 用户注册 Service 实现方法

```

<!-- 按用户名查用户 -->
<select id="findUserByUsername" resultMap="User">
    select * from user where username = #{username}
</select>
<!-- 查询是否存在 -->
<select id="exists">
    select exists (select id from user where username = #{username})
</select>
<!-- 查询用户总数 -->
<select id="count">
    select count(id) from user
    <if test="name != null and name != ''">
        where username like concat('%',#{name,jdbcType=VARCHAR},'%')
    </if>
</select>
<!-- 查询用户总数 -->
<select id="findUserById" resultMap="User">
    select * from user where id = #{userId}
</select>
<!-- 查询用户的收藏列表 -->
<select id="selectPage" resultMap="User">
    select * from user
    <if test="name != null and name != ''">
        where username like concat('%',#{name,jdbcType=VARCHAR},'%')
    </if>
    order by create_time desc
    <if test="startPosition != null and pageSize != null">
        limit #{startPosition},#{pageSize}
    </if>
</select>
<!-- 保存用户 -->
<select id="save" parameterType="User">
    insert into user(id, username, password, create_time, update_time, enabled, is_admin)
    values(null, #{username}, #{password}, #{createTime}, #{updateTime}, #{enabled}, #{isAdmin})
</select>

```

图附录-6 用户表 SQL 代码 1

```

<!-- 更新用户 -->
<update id="update">
    update user
    <set>
        <if test='username != null and username != ""'>
            username = #{username},
        </if>
        <if test="password != null">
            password = #{password},
        </if>
        <if test="createTime != null">
            create_time = #{createTime},
        </if>
        <if test="updateTime != null">
            update_time = #{updateTime},
        </if>
        <if test="enabled != null">
            enabled = #{enabled},
        </if>
        <if test="isAdmin != null">
            is_admin = #{isAdmin},
        </if>
    </set>
    where id = #{id}
</update>
<!-- 删除用户 -->
<delete id="delete" parameterType="java.lang.Integer">
    delete from user where id = #{userId}
</delete>

```

图附录-7 用户表 SQL 代码 2

```

@Override
public Category findOne(Integer id) {
    return categoryMapper.select(id);
}

@Override
public Category getCategoryByName(String categoryName) {
    return categoryMapper.selectCategoryByName(categoryName);
}

@Override
public List<Category> listAll() {
    return categoryMapper.selectAll();
}

@Override
public Page<List<Category>> listPage(Integer currPage, Integer pageSize) {
    Page<List<Category>> page = new Page<>();
    Integer totalCount = categoryMapper.count();
    Integer startPosition = Page.calculateStartPosition(currPage, pageSize);
    List<Category> categorys = categoryMapper.selectPage(startPosition, pageSize);
    page.setCount(totalCount);
    page.setCurr(currPage);
    page.setLimit(pageSize);
    page.setData(categorys);
    return page;
}

@Override
@Transactional(rollbackFor = Exception.class)
public void save(Category category) {
    if (category.getId() != null) {
        throw new BusinessException("保存时不能带上ID");
    }
    categoryMapper.save(category);
}

```

图附录-8 分类 Service 实现方法 1

```

@Override
@Transactional(rollbackFor = Exception.class)
public void save(Category category) {
    if (category.getId() != null) {
        throw new BusinessException("保存时不能带上ID");
    }
    categoryMapper.save(category);
}

@Override
@Transactional(rollbackFor = Exception.class)
public void update(Category category) {
    if (category.getId() == null) {
        throw new BusinessException("更新时必须带上ID");
    }
    Category select = categoryMapper.select(category.getId());
    if (select != null) {
        categoryMapper.update(category);
    } else {
        throw new BusinessException("待更新分类不存在");
    }
}

@Override
@Transactional(rollbackFor = Exception.class)
public void delete(Integer id) {
    Category select = categoryMapper.select(id);
    if (select != null) {
        categoryMapper.delete(id);
    }
}

```

图附录-9 分类 Service 实现方法 2

```

<!-- 按分类id查图书分类 -->
<select id="select" resultMap="Category">
    select * from category where id = #{id}
</select>
<!-- 按分类名查图书分类 -->
<select id="selectCategoryByName" resultMap="Category">
    select * from category where name = #{categoryName}
</select>
<!-- 查询所有图书分类 -->
<select id="selectAll" resultMap="Category">
    select * from category order by id asc
</select>
<!-- 查询图书分类总数 -->
<select id="count" resultType="java.lang.Integer">
    select count(id) from category
</select>
<!-- 分页查找图书分类列表 -->
<select id="selectPage" resultMap="Category">
    select * from category order by id asc
    <if test="currPosition != null and pageSize != null">
        limit #{currPosition},#{pageSize}
    </if>
</select>
<!-- 插入图书分类 -->
<insert id="save" parameterType="Category">
    insert into category(id, name) values(null, #{name})
</insert>
<!-- 更新图书分类 -->
<update id="update" parameterType="Category">
    update category
    <set>
        <if test="name != null">
            name = #{name},
        </if>
    </set>
    where id = #{id}
</update>
<!-- 删除图书分类 -->
<delete id="delete">
    delete from category where id = #{id}
</delete>

```

图附录-10 分类表 SQL 代码

```

public Page<List<BookItemDTO>> search(String keyword, String categoryTitle, Integer currPage, Integer pageSize) {
    logger.info("图书搜索方法执行. 入参: keyword={},categoryTitle={},currPage={},pageSize={}", keyword, categoryTitle,
        currPage, pageSize);
    Page<List<BookItemDTO>> page = new Page<>();
    // 总数据量
    Integer count;
    // 分页起始位置
    Integer startPosition = Page.calculateStartPosition(currPage, pageSize);
    // 图书列表
    List<Book> books;
    if (StringUtils.isNotBlank(keyword)) {
        // 按书名和isbn查
        String isbn = keyword.replace("_", "");
        logger.info("使用关键词查询图书列表", keyword, isbn);
        count = bookMapper.findBookCountByKeyword(keyword, isbn);
        books = bookMapper.findBooksByKeyword(keyword, isbn, startPosition, pageSize);
    } else {
        // 按分类标题查
        logger.info("使用分类标题进行查找图书");
        Category category = categoryService.getCategoryByName(categoryTitle);
        if (category == null) {
            throw new BusinessException("系统无名为“" + categoryTitle + ”的分类");
        }
        count = bookMapper.findBookCountByCategoryId(category.getId());
        books = bookMapper.findBooksByCategoryId(category.getId(), startPosition, pageSize);
    }

    List<Category> categorys = categoryService.listAll();
    Map<Integer, String> categoryMap = categorys.stream()
        .collect(Collectors.toMap(Category::getId, Category::getName));

    List<BookItemDTO> bookItems = new ArrayList<>();
    if (books != null) {
        BookItemDTO temp;
        for (Book b : books) {
            temp = new BookItemDTO();
            BeanUtils.copyProperties(b, temp);
            temp.setCategoryName(categoryMap.get(temp.getCategoryId()));
            bookItems.add(temp);
        }
    }

    page.setCount(count);
    page.setCurr(currPage);
    page.setLimit(pageSize);
    page.setData(bookItems);
    return page;
}

```

图附录-11 图书 Service 实现方法 1

```

public void save(Book book) {
    Book exists = bookMapper.findBookByName(book.getName());
    if (exists != null) {
        throw new BusinessException("该书已添加, 请勿重复上架");
    }
    bookMapper.save(book);
}

@Override
public void update(Book book) {
    Book findBookById = bookMapper.findBookById(book.getId());
    if (findBookById != null) {
        findBookById.setName(book.getName());
        findBookById.setAuthor(book.getAuthor());
        findBookById.setIntro(book.getIntro());
        findBookById.setCover(book.getCover());
        findBookById.setPress(book.getPress());
        findBookById.setPublicationDate(book.getPublicationDate());
        findBookById.setIsbn(book.getIsbn());
        findBookById.setCategoryId(book.getCategoryId());
        findBookById.setPdfUrl(book.getPdfUrl());
        findBookById.setTotalShelves(book.getTotalShelves());
        findBookById.setUpdateTime(LocalDateTime.now());
        bookMapper.update(findBookById);
        return;
    }
    throw new BusinessException("修改的图书未找到");
}

```

图附录-12 图书 Service 实现代码 2

```

public void delete(int bookId) {
    Book findBookById = bookMapper.findBookById(bookId);
    if (findBookById != null) {
        // 删除图片、pdf、图书记录
        deleteOssObject(findBookById.getPdfUrl());
        deleteOssObject(findBookById.getCover());
        bookMapper.delete(bookId);
    }
}

private void deleteOssObject(String fullUrl) {
    minioService.deleteObject(minioBucketName, fullUrl.replace(Constants.DOWNLOAD_PREFIX, ""));
}

@Override
public Book findBookById(Integer bookId) {
    return bookMapper.findBookById(bookId);
}

@Override
public List<Book> findBooks(List<Integer> bookIds) {
    return bookMapper.batchGetBooks(bookIds);
}

@Override
public Page<List<Book>> findBooksPageble(Integer currPage, Integer pageSize, String name) {
    Page<List<Book>> page = new Page<>();
    Integer totalCount = bookMapper.count(name);
    Integer startPosition = Page.calculateStartPosition(currPage, pageSize);
    List<Book> books = bookMapper.selectPage(name, startPosition, pageSize);
    page.setCount(totalCount);
    page.setCurr(currPage);
    page.setLimit(pageSize);
    page.setData(books);
    return page;
}

@Override
public Page<List<Book>> findBookListByCategoryId(Integer categoryId, Integer currPage, Integer pageSize) {
    Page<List<Book>> page = new Page<>();
    Integer count = bookMapper.findBookCountByCategoryId(categoryId);
    Integer startPosition = Page.calculateStartPosition(currPage, pageSize);
    List<Book> books = bookMapper.findBooksByCategoryId(categoryId, startPosition, pageSize);
    page.setCount(count);
    page.setCurr(currPage);
    page.setLimit(pageSize);
    page.setData(books);
    return page;
}
}

```

图附录-13 图书 Service 实现代码 3

```

<!-- 按id查询图书 -->
<select id="findBookById" resultMap="Book">
    select * from book where id = #{id}
</select>
<!-- 查询时锁定图书信息，加写锁 -->
<select id="findBookForUpdate" resultMap="Book">
    select * from book where id = #{id} for update
</select>
<!-- 按关键字查找符合isbn或书名的数量 -->
<select id="findBookCountByKeyword" resultType="java.lang.Integer">
    select count(1) from book where isbn = #{isbn} or name like concat('%',#{keyword,jdbcType=VARCHAR},'%')
</select>
<!-- 按关键字查找符合isbn或书名的图书列表 -->
<select id="findBooksByKeyword" resultMap="Book">
    select * from book where isbn = #{isbn} or name like concat('%',#{keyword,jdbcType=VARCHAR},'%')
    <if test="currPosition != null and pageSize != null">
        limit #{currPosition},#{pageSize}
    </if>
</select>
<!-- 按图书id列表批量查图书列表 -->
<select id="batchGetBooks" resultMap="Book">
    select * from book where id in
    <foreach collection="bookIds" item="bookId" open="(" separator="," close=")">
        #{bookId}
    </foreach>
</select>

```

图附录-14 图书表 SQL 代码 1

```

<!-- 按分类id查找图书数量 -->
<select id="findBookCountByCategoryId" resultType="java.lang.Integer">
    select count(1) from book where category_id = #{categoryId}
</select>
<!-- 获取图书总量 -->
<select id="count" resultType="java.lang.Integer">
    select count(name) from book
    <if test="name != null and name != ''">
        where name like concat('%',#{name,jdbcType=VARCHAR},'%')
    </if>
</select>
<!-- 分页查询图书列表 -->
<select id="selectPage" resultMap="Book">
    select * from book
    <if test="name != null and name != ''">
        where name like concat('%',#{name,jdbcType=VARCHAR},'%')
    </if>
    order by create_time desc
    <if test="startPosition != null and pageSize != null">
        limit #{startPosition},#{pageSize}
    </if>
</select>
<!-- 按分类获取图书列表 -->
<select id="findBooksByCategoryId" resultMap="Book">
    select * from book where category_id = #{categoryId}
    <if test="currPosition != null and pageSize != null">
        limit #{currPosition},#{pageSize}
    </if>
</select>
<!-- 按名称查图书 -->
<select id="findBookByName" resultMap="Book">
    select * from book where name = #{bookName}
</select>
<!-- 更新图书借阅数 -->
<update id="updateBookBorrowCount">
    update book set borrow_count = #{borrowCount} where id = #{id}
</update>

```

图附录-15 图书表 SQL 代码 2

```

<!-- 插入图书 -->
<insert id="save" parameterType="Book">
    insert into book(id,name,author,intro,cover,press,publication_date,isbn,category_id,pdf_url,total_shelves,borrow_count,create_time,update_time)
    values(null,#{name},#{author},#{intro},#{cover},#{press},#{publicationDate},#{isbn},#{categoryId},#{pdfUrl},#{totalShelves},#{borrowCount},#{createTime},
</insert>

```

图附录-16 图书表 SQL 代码 3

```

<!-- 更新图书 -->
<update id="update" parameterType="Book">
    update book
    <set>
        <if test="name != null">
            name=#{name},
        </if>
        <if test="author != null">
            author=#{author},
        </if>
        <if test="intro != null">
            intro=#{intro},
        </if>
        <if test="cover != null">
            cover=#{cover},
        </if>
        <if test="press != null">
            press=#{press},
        </if>
        <if test="publicationDate != null">
            publication_date=#{publicationDate},
        </if>
        <if test="isbn != null">
            isbn=#{isbn},
        </if>
        <if test="categoryId != null">
            category_id=#{categoryId},
        </if>
        <if test="pdfUrl != null">
            pdf_url=#{pdfUrl},
        </if>
        <if test="totalShelves != null">
            total_shelves=#{totalShelves},
        </if>
        <if test="borrowCount != null">
            borrow_count=#{borrowCount},
        </if>
        <if test="createTime != null">
            create_time=#{createTime},
        </if>
        <if test="updateTime != null">
            update_time=#{updateTime},
        </if>
    </set>
    where id = #{id}
</update>
<!-- 删除图书 -->
<delete id="delete">
    delete from book where id = #{id}
</delete>

```

图附录-17 图书表 SQL 代码 4

```

@Override
@Transactional(rollbackFor = Exception.class)
public void borrowBook(Integer bookId, Integer userId) {
    Boolean recordExists = borrowRecordMapper.existsUnReturnBookBorrowRecord(bookId, userId);
    if (recordExists) {
        throw new BusinessException("已有借阅记录, 请勿重复借阅");
    }
    Book origin = bookMapper.findBookForUpdate(bookId);
    int total = origin.getTotalShelves();
    int borrowCount = origin.getBorrowCount();
    if (total - borrowCount <= 0) {
        throw new BusinessException("已无库存");
    }
    int updateRow = bookMapper.updateBookBorrowCount(origin.getId(), borrowCount + 1);
    if (updateRow > 0) {
        borrowRecordMapper.saveBorrowRecord(getBorrowRecord(bookId, userId));
    } else {
        throw new BusinessException("扣库存失败");
    }
}

private BorrowRecord getBorrowRecord(Integer bookId, Integer userId) {
    BorrowRecord record = new BorrowRecord();
    record.setBookId(bookId);
    record.setUserId(userId);
    LocalDateTime now = LocalDateTime.now();
    record.setBorrowTime(now);
    record.setForceReturnTime(now.plusDays(5));
    record.setStatus(Consts.NO_RETURN_YET);
    return record;
}

```

图附录-18 借阅 Service 实现代码 1

```

@Override
@Transactional(rollbackFor = Exception.class)
public void returnBook(Integer bookId, Integer userId) {
    Boolean recordExists = borrowRecordMapper.existsUnReturnBookBorrowRecord(bookId, userId);
    if (!recordExists) {
        throw new BusinessException("没有借阅记录, 非法操作");
    }
    List<BorrowRecord> unReturnBookBorrowRecords = borrowRecordMapper.selectUnReturnBookBorrowRecords(bookId,
        userId);
    if (unReturnBookBorrowRecords == null || unReturnBookBorrowRecords.isEmpty()) {
        throw new BusinessException("不存在或存在多条同名图书借阅记录, 数据不正确");
    }
    // 加锁读图书, 还库存
    Book findBookForUpdate = bookMapper.findBookForUpdate(bookId);
    findBookForUpdate.setBorrowCount(findBookForUpdate.getBorrowCount() - 1);
    bookMapper.update(findBookForUpdate);

    // 更新借阅记录还书时间、状态
    BorrowRecord borrowRecord = unReturnBookBorrowRecords.get(0);
    borrowRecordMapper.updateBorrowRecordReturnState(borrowRecord.getId(), Consts.RETURNED, LocalDateTime.now());
}

@Override
public String getUnReturnedRecordForceReturnTime(Integer bookId, Integer userId) {
    LocalDateTime forceReturnTime = borrowRecordMapper.getUnReturnRecordForceReturnTime(bookId, userId);
    // 有待还记录, 返回强制还书时间字符串; 没有的返回当前时间
    LocalDateTime returnDate;
    if (forceReturnTime != null) {
        returnDate = forceReturnTime;
    } else {
        returnDate = LocalDateTime.now();
    }
    return returnDate.format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm"));
}

@Override
public Boolean bookBorrowRecordExists(Integer bookId, Integer userId, Integer status) {
    return borrowRecordMapper.existsBookStatusBorrowRecord(bookId, userId, status);
}

```

图附录-19 借阅 Service 实现代码 2

```

@Override
public Page<List<BorrowRecord>> listPage(Integer userId, Integer currPage, Integer pageSize) {
    Page<List<BorrowRecord>> page = new Page<>();
    Integer totalCount = borrowRecordMapper.selectUserBorrowRecordSum(userId);
    Integer startPosition = Page.calculateStartPosition(currPage, pageSize);
    List<BorrowRecord> borrowRecords = borrowRecordMapper.selectPage(userId, startPosition, pageSize);
    page.setCount(totalCount);
    page.setCurr(currPage);
    page.setLimit(pageSize);
    page.setData(borrowRecords);
    return page;
}

@Override
@Transactional(rollbackFor = Exception.class)
public Integer forceReturnBook() {
    List<BorrowRecord> borrowRecords = borrowRecordMapper.selectAllUnReturnOutDateBorrowRecords();
    if (borrowRecords == null || borrowRecords.isEmpty()) {
        return 0;
    }
    Book book = null;
    for (BorrowRecord borrowRecord : borrowRecords) {
        book = bookMapper.findBookById(borrowRecord.getBookId());
        if (book != null) {
            // 加行锁
            Book findBookForUpdate = bookMapper.findBookForUpdate(book.getId());
            findBookForUpdate.setBorrowCount(findBookForUpdate.getBorrowCount() - 1);
            bookMapper.update(findBookForUpdate);
            // 更新还书状态
            borrowRecordMapper.updateBorrowRecordReturnState(borrowRecord.getId(), Consts.FORCE_RETURNED,
                LocalDateTime.now());
        }
    }
    return borrowRecords.size();
}

```

图附录-20 借阅 Service 实现代码 3

```

<!-- 获取待还书籍的强制还书时间 -->
<select id="getUnReturnRecordForceReturnTime" resultType="java.time.LocalDateTime">
    select force_return_time from borrow_record
    where user_id = #{userId} and book_id = #{bookId}
    and status = 0
</select>
<!-- 查看用户是否有未还指定图书的记录 -->
<select id="existsUnReturnBookBorrowRecord">
    select exists(select * from borrow_record
    where user_id = #{userId} and book_id = #{bookId}
    and status = 0)
</select>
<!-- 查看用户未还指定图书的记录 -->
<select id="selectUnReturnBookBorrowRecords" resultMap="BorrowRecord">
    select * from borrow_record
    where user_id = #{userId} and book_id = #{bookId}
    and status = 0
</select>
<!-- 查看用户是否有指定状态的图书 -->
<select id="existsBookStatusBorrowRecord">
    select exists(select * from borrow_record
    where user_id = #{userId} and book_id = #{bookId}
    and status = #{status})
</select>
<!-- 保存借阅记录 -->
<insert id="saveBorrowRecord" parameterType="BorrowRecord">
    insert into
    borrow_record(id,user_id,book_id,borrow_time,force_return_time,return_time,status)
    values(null, #{userId}, #{bookId}, #{borrowTime}, #{forceReturnTime},
    #{returnTime}, #{status})
</insert>

```

图附录-21 借阅表 SQL 代码 1

```

<!-- 根据id更新借阅记录状态及还书时间 -->
<update id="updateBorrowRecordReturnState">
    update borrow_record
    <set>
        <if test="status != null">
            status = #{status},
        </if>
        <if test="returnTime != null">
            return_time = #{returnTime},
        </if>
    </set>
    where id = #{id}
</update>
<!-- 查询用户的借阅记录数 -->
<select id="selectUserBorrowRecordSum" parameterType="java.lang.Integer" resultType="java.lang.Integer">
    select count(id) from borrow_record
    where user_id=#{userId}
</select>
<!-- 查询用户的借阅记录列表 -->
<select id="selectPage" resultMap="BorrowRecord">
    select * from borrow_record where user_id=#{userId}
    order by borrow_time desc
    <if test="startPosition != null and pageSize != null">
        limit #{startPosition},#{pageSize}
    </if>
</select>
<!-- 查询所有未还过期借阅记录 -->
<select id="selectAllUnReturnOutDateBorrowRecords" resultMap="BorrowRecord">
    select * from borrow_record
    where status = 0 and curtime() > force_return_time
</select>

```

图附录-22 借阅表 SQL 代码 2

```

@Override
public void collect(Integer bookId, Integer userId) {
    Collections c = new Collections();
    c.setUserId(userId);
    c.setBookId(bookId);
    c.setCreateTime(LocalDateTime.now());
    collectionMapper.save(c);
}

@Override
public void cancel(Integer bookId, Integer userId) {
    Boolean exists = exists(bookId, userId);
    if (exists) {
        Collections c = new Collections();
        c.setUserId(userId);
        c.setBookId(bookId);
        collectionMapper.delete(c);
    }
}

@Override
public Boolean exists(Integer bookId, Integer userId) {
    return collectionMapper.exists(bookId, userId);
}

@Override
public Integer bookCollectSum(Integer bookId) {
    return collectionMapper.selectBookCollectSum(bookId);
}

@Override
public Page<List<Collections>> listPage(Integer userId, Integer currPage, Integer pageSize) {
    Page<List<Collections>> page = new Page<>();
    Integer totalCount = collectionMapper.selectUserCollectSum(userId);
    Integer startPosition = Page.calculateStartPosition(currPage, pageSize);
    List<Collections> collections = collectionMapper.selectPage(userId, startPosition, pageSize);
    page.setCount(totalCount);
    page.setCurr(currPage);
    page.setLimit(pageSize);
    page.setData(collections);
    return page;
}

```

图附录-23 收藏 Service 实现代码

```

<!-- 查询收藏是否存在 -->
<select id="exists" resultType="java.lang.Boolean">
    select exists (select 1 from collection
        where user_id = #{userId} and book_id=#{bookId})
</select>
<!-- 查询图书的收藏数 -->
<select id="selectBookCollectSum" resultType="java.lang.Integer">
    select count(id) from collection
    where book_id=#{bookId}
</select>
<!-- 查询用户的图书收藏数 -->
<select id="selectUserCollectSum" parameterType="java.lang.Integer" resultType="java.lang.Integer">
    select count(id) from collection
    where user_id=#{userId}
</select>
<!-- 查询用户的收藏列表 -->
<select id="selectPage" resultMap="Collections">
    select * from collection where user_id=#{userId}
    order by create_time desc
    <if test="startPosition != null and pageSize != null">
        limit #{startPosition},#{pageSize}
    </if>
</select>
<!-- 添加收藏 -->
<insert id="save" parameterType="Collections">
    insert into collection(id, user_id, book_id, create_time)
    values(null, #{userId}, #{bookId}, #{createTime})
</insert>
<!-- 删除收藏 -->
<delete id="delete" parameterType="Collections">
    delete from collection
    <where>
        <if test="id != null">
            id = #{id},
        </if>
        <if test="bookId != null and userId != null">
            book_id = #{bookId} and user_id = #{userId}
        </if>
    </where>
</delete>

```

图附录-24 收藏表 SQL 代码

```

private MinioClient minioClient;

public MinioServiceImpl(@Value("${minio.endpoint}") String endpoint, @Value("${minio.accessKey}") String accessKey,
    @Value("${minio.secretKey}") String secretKey) throws MalformedURLException {
    URL endpointUrl = new URL(endpoint);
    try {
        // url上无端口号时, 识别http为80端口, https为443端口
        int port = endpointUrl.getPort() != -1 ? endpointUrl.getPort() : endpointUrl.getDefaultPort();
        boolean security = HTTP_PROTOCOL.equals(endpointUrl.getProtocol()) ? false : true;
        // @formatter:off
        this.minioClient = MinioClient.builder().endpoint(endpointUrl.getHost(), port, security)
            .credentials(accessKey, secretKey).build();
        // @formatter:on
        // 忽略证书校验, 防止自签名证书校验失败导致无法建立连接
        this.minioClient.ignoreCertCheck();
    } catch (Exception e) {
        e.printStackTrace();
    }
}

@Override
public boolean createBucket(String bucketName) {
    try {
        boolean found = minioClient.bucketExists(BucketExistsArgs.builder().bucket(bucketName).build());
        if (found) {
            logger.info("bucket {} already exists.", bucketName);
            return true;
        }
        minioClient.makeBucket(MakeBucketArgs.builder().bucket(bucketName).build());
        return true;
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}

```

图附录-25 MinIO 上传下载 Service 代码 1

```

public boolean bucketExists(String bucketName) {
    try {
        return minioClient.bucketExists(BucketExistsArgs.builder().bucket(bucketName).build());
    } catch (Exception e) {
        e.printStackTrace();
    }
    return false;
}

@Override
public void upload(String bucketName, String objectId, InputStream input) {
    try {
        //@formatter:off
        minioClient.putObject(PutObjectArgs.builder().bucket(bucketName).object(objectId)
            .stream(input, input.available(), -1)
            .build());
        //@formatter:on
    } catch (Exception e) {
        e.printStackTrace();
    }
}

@Override
public InputStream download(String bucketName, String objectId) {
    try {
        return minioClient.getObject(GetObjectArgs.builder().bucket(bucketName).object(objectId).build());
    } catch (Exception e) {
        e.printStackTrace();
    }
    return null;
}

```

图附录-26 MinIO 上传下载 Service 代码 2

```

@Override
public boolean deleteObject(String bucketName, String objectId) {
    //@formatter:off
    try {
        minioClient.removeObject(RemoveObjectArgs.builder()
            .bucket(bucketName).object(objectId).build());
    } catch (Exception e) {
        e.printStackTrace();
    }
    //@formatter:on
    return true;
}

@Override
public boolean objectExists(String bucketName, String objectId) {
    //@formatter:off
    try {
        // minio客户端未提供判断对象是否存在的方法，此方法中调用出现异常时说明对象不存在
        minioClient.statObject(StatObjectArgs.builder()
            .bucket(bucketName).object(objectId).build());
    } catch (Exception e) {
        return false;
    }
    //@formatter:on
    return true;
}

```

图附录-27 MinIO 上传下载 Service 代码 3

```

layui.use(function() {
    var form = layui.form;
    var layer = layui.layer;
    var $ = layui.$;
    // 提交事件
    form.on('submit(login)', function(data) {
        var field = data.field; // 获取表单字段值
        $.ajax({
            url: "/user/login",
            data: JSON.stringify(field),
            type: "POST",
            dataType: "json",
            contentType: "application/json",
            success: function(data) {
                console.log("data=>", data);
                if (data.code === 200) {
                    layer.msg('登录成功', {
                        icon: 1,
                        time: 500 // 设置0.5秒后自动关闭
                    }, function() {
                        // 先得到当前 iframe 层的索引
                        var index = parent.layer.getFrameIndex(window.name);
                        // 再执行关闭
                        parent.layer.close(index);
                    });
                } else {
                    layer.msg("登录失败，原因：" + data.msg);
                }
            }
        });
        return false; // 阻止默认 form 跳转
    });
});

```

图附录-28 登录 JS 代码

```

//加载时执行，如会话中存在用户即更新用户名及添加注销按钮
var $ = layui.$;
$.getJSON('/user/current', function(res) {
    if (res.code === 200) {
        updateMenu(layui, res.data);
    }
});
//更新登录用户名dom
function updateMenu(layui, data) {
    var $ = layui.$;
    var element = layui.element;
    if ($("#login-tab").text().includes("登录") === true) {
        $("#login-tab").text(data.username);
        var html =
            '<dl class="layui-nav-child"><dd style="text-align: center;"><a href="/user/logout">注销</a></dd></dl>';
        $(".login-tab-group").append(html);
    }
    //取出菜单数量
    var menuSize = $(".layui-nav.layui-layout-right li").size();
    // 管理员要显示的菜单
    if (menuSize == 3) { //防止重复追加菜单
        var menus = '';
        if (data.admin) {
            menus = '<li class="layui-nav-item"><a href="/html/category-manage.html">分类管理</a></li>' +
                '<li class="layui-nav-item"><a href="/html/book-manage.html">图书管理</a></li>' +
                '<li class="layui-nav-item"><a href="/html/user-manage.html">用户管理</a></li>';
        } else {
            menus = '<li class="layui-nav-item"><a href="javascript:categorySearch()">分类查询</a></li>' +
                '<li class="layui-nav-item"><a href="/html/my-borrow.html">我的借阅</a></li>' +
                '<li class="layui-nav-item"><a href="/html/my-collection.html">我的收藏</a></li>';
        }
        $("#index-page-btn").after(menus);
        element.render('nav', 'menu-filter-nav');
    }
}

```

图附录-29 导航栏 JS 代码 1

```

function categorySearch() {
    if ($("#login-tab").text().includes("登录") === true) {
        layui.layer.msg("请先登录再尝试!");
    } else {
        location.href = '/html/search.html?categoryTitle=计算机';
    }
}

//登录注册弹窗
layui.use(['element', 'layer', 'util'], function() {
    var layer = layui.layer;
    var util = layui.util;
    var element = layui.element;
    var $ = layui.$;
    util.on('lay-on', {
        iframe: function() {
            // 先判断会话是否已经登录
            $.getJSON('/user/current', function(res) {
                console.log("res->", res);
                if (res.code === 200) {
                    updateMenu(layui, res.data);
                    return;
                }
                // 仅未登录需要弹窗
                var index = layer.open({
                    type: 2,
                    title: '登录/注册',
                    shadeClose: true,
                    shade: 0.8,
                    area: ['380px', '320px'],
                    content: '/html/login.html',
                    end: function() {
                        //刷新当前页面
                        location.reload();
                    }
                });
            });
        }
    });
});
element.render('nav', 'menu-filter-nav');
});

```

图附录-30 导航栏 JS 代码 2

```

// 自定义验证规则
form.verify({
    // 确认密码
    confirmPassword: function(value, item) {
        var passwordValue = $('#reg-password').val();
        if (value !== passwordValue) {
            return '两次密码输入不一致';
        }
    }
});

// 提交事件
form.on('submit(register)', function(data) {
    var field = data.field; // 获取表单字段值
    $.ajax({
        url: "/user/register",
        data: JSON.stringify(field),
        type: "POST",
        dataType: "json",
        contentType: "application/json",
        success: function(data) {
            var res = data.data;
            if (data.code === 200) {
                layer.msg('注册成功', {
                    icon: 1,
                    time: 1000 // 设置1秒后自动关闭
                }, function() {
                    //倒计时结束后跳转登录页
                    location.href = "/html/login.html";
                });
            } else {
                layer.msg(data.msg);
            }
        }
    });
    return false; // 阻止默认 form 跳转
});

```

图附录-31 用户注册 JS 代码

```

layui.use(function() {
  var form = layui.form;
  var layer = layui.layer;
  var $ = layui.$;
  // 提交事件
  form.on('submit(login)', function(data) {
    var field = data.field; // 获取表单字段值
    $.ajax({
      url: "/user/login",
      data: JSON.stringify(field),
      type: "POST",
      dataType: "json",
      contentType: "application/json",
      success: function(data) {
        console.log("data=>", data);
        if (data.code === 200) {
          layer.msg('登录成功', {
            icon: 1,
            time: 500 // 设置0.5秒后自动关闭
          }, function() {
            // 先得到当前 iframe 层的索引
            var index = parent.layer.getFrameIndex(window.name);
            // 再执行关闭
            parent.layer.close(index);
          });
        } else {
          layer.msg("登录失败，原因: " + data.msg);
        }
      }
    });
    return false; // 阻止默认 form 跳转
  });
});

```

图附录-32 用户登录 JS 代码

```

// 输入框点缀事件
form.on('input-affix(search)', function(data) {
  console.log("data", data)
  var elem = data.elem; // 输入框
  var keyword = elem.value; // 输入框的值
  if (!keyword) {
    layer.msg('请输入搜索内容');
    return elem.focus()
  };
  // 搜索跳转
  jumpSearch(keyword)
});
// 菜单点击事件
dropdown.on('click(book-category)', function(options) {
  location.href = '/html/search.html?categoryTitle=' + options.title;
});
// 设置页首关键词或分类字样
var keyword = getKeyword();
if (keyword !== '') {
  $("#keywordType").text('关键词: ');
  $("#keyword").text(decodeURI(keyword));
} else {
  $("#keywordType").text('分类名: ');
  $("#keyword").text(decodeURI(getCategoryTitle()));
}
var inst = table.render({
  elem: '#books-table',
  url: '/book/search?keyword=' + getKeyword() + '&categoryTitle=' + getCategoryTitle(),
  cols: [

```

图附录-33 图书搜索 JS 代码

```

//编辑 或 删除 或 设为管理员
table.on('tool(user-table)', function(obj) {
    var data = obj.data; // 获得当前行数据
    if (obj.event === 'toggleEnabled') {
        $.ajax({
            url: "/user/toggle?userId=" + data.id + '&option=enabled',
            type: 'POST',
            data: {},
            success: function(d) {
                if (d.code === 200) {
                    table.reloadData('userTable');
                } else {
                    layer.msg('切换失败, ' + d.msg);
                }
            },
            dataType: "json",
            contentType: "application/json"
        });
    } else if (obj.event === 'toggleManage') {
        $.ajax({
            url: "/user/toggle?userId=" + data.id + '&option=managed',
            type: 'POST',
            data: {},
            success: function(d) {
                if (d.code === 200) {
                    table.reloadData('userTable');
                } else {
                    layer.msg('切换失败, ' + d.msg);
                }
            },
            dataType: "json",
            contentType: "application/json"
        });
    } else if (obj.event === 'delete') {
        $.ajax({
            url: "/user/delete?userId=" + data.id,
            type: 'DELETE',
            data: {},
            success: function(d) {
                if (d.code === 200) {
                    table.reloadData('userTable');
                } else {
                    layer.msg('删除失败, 原因: ' + d.msg);
                }
            },
            dataType: "json",
            contentType: "application/json"
        });
    }
});
});

```

图附录-34 用户管理 JS 代码 1

```

var $ = layui.$;
function getUsername() {
    var strArray = location.href.split("?username=");
    if (strArray !== undefined && strArray.length >= 2) {
        return strArray[1];
    }
    return '';
}
//搜索用户
function jumpSearch(username) {
    location.href = '/html/user-manage.html?username=' + username;
}
layui.use(['table', 'layer'], function() {
    var $ = layui.jquery;
    var table = layui.table;
    var layer = layui.layer;

    // 渲染, 并获得实例对象
    table.render({
        elem: '#user-table', //表格ID
        toolbar: "#tableToolbar", //表格工具栏ID
        url: '/user/listPage?name=' + getUsername(),
        id: 'userTable', //给表格加一个ID标识, 便于重载数据
        defaultToolbar: [],
    });
});

```

图附录-35 用户管理 JS 代码 2

```

layui.util.on('lay-on', {
  // 输入框搜索按钮触发
  "search-book": function() {
    var text = $("#search-input").val();
    jumpSearch(text);
  }
});
table.on('toolbar(book-table)', function(obj) {
  console.log("event:", obj)
  if (obj.event === 'add') {
    // 打开添加分类iframe
    layer.open({
      type: 2,
      title: "添加图书", //标题
      content: "/html/add-book.html",
      area: ["500px", "720px"],
      end: function() {
        table.reloadData('bookTable');
        console.log("添加图书iframe关闭，刷新数据已执行");
      }
    });
  } else if (obj.event === 'refresh') {
    console.log("refresh事件执行了")
    table.reloadData('bookTable');
  }
});
//编辑 或 下架图书项
table.on('tool(book-table)', function(obj) { // 双击 toolDouble
  var data = obj.data; // 获得当前行数据
  if (obj.event === 'edit') {
    // 打开修改分类iframe
    layer.open({
      type: 2,
      title: "修改图书",
      content: "/html/edit-book.html?id=" + data.id,
      area: ["500px", "720px"], //指定弹层宽高
      end: function() {
        table.reloadData('bookTable');
        console.log("添加分类iframe关闭，刷新数据已执行");
      }
    });
  } else if (obj.event === 'delete') {
    $.ajax({
      url: "/book/delete?id=" + data.id,
      type: 'DELETE',
      data: {},
      success: function(d) {
        if (d.code === 200) {
          table.reloadData('bookTable');
        } else {
          layer.msg('删除失败，原因: ' + d.msg);
        }
      }
    });
  },

```

图附录-36 图书管理 JS 代码 1

```

//搜索图书
function jumpSearch(bookName) {
  location.href = '/html/book-manage.html?name=' + bookName;
}
layui.use(['table', 'layer', 'form', 'util'], function() {
  var table = layui.table;
  var layer = layui.layer;
  var form = layui.form;
  // 渲染表格
  table.render({
    elem: '#book-table', //表格ID
    toolbar: "#tableToolbar", //表格工具栏ID
    defaultToolbar: [],
    height: 'auto',
    url: '/book/listPage?name=' + getBookName(),
    id: 'bookTable', //给表格加一个ID标识，便于重载数据

```

图附录-37 图书管理 JS 代码 2

```

layui.use(['table', 'layer'], function() {
  var $ = layui.jquery;
  var table = layui.table;
  var layer = layui.layer;

  // 渲染, 并获得实例对象
  table.render({
    elem: '#category-table', //表格ID
    toolbar: "#tableToolbar", //表格工具栏ID
    url: '/category/listPage',
    id: 'categoryTable', //给表格加一个ID标识, 便于重载数据
    defaultToolbar: [],
    cols: [
      [
        {
          field: "categoryId",
          title: "分类ID",
          sort: true,
          templet: function(d) {
            return d.id;
          }
        },
        {
          field: "categoryName",
          title: "分类名称",
          sort: true,
          templet: function(d) {
            return d.name;
          }
        },
        {
          field: "operations",
          title: "操&emsp;&emsp;作",
          align: "center",
          templet: function(d) {
            var buttonGroup = '<div class="layui-clear-space">' +
              '<a class="layui-btn layui-btn-xs" lay-event="edit">编辑</a>' +
              '<a class="layui-btn layui-btn-xs layui-btn-danger" lay-event="delete">删除</a>' +
              '</div>';
            return buttonGroup;
          }
        }
      ]
    ],
    parseData: function(res) { //请求响应体与layui不同时, 使用此方法转换
      return {
        "code": res.code == 200 ? 0 : res.code,
        "msg": res.msg,
        "count": res.data.count,
        "data": res.data.data
      }
    },
    page: true, //开启分页
    limit: 5, //每页默认显示的数量
    limits: [5, 10] //每页可选的数量
  });
});

```

图附录-38 分类管理 JS 代码 1

```

table.on('toolbar(category-table)', function(obj) {
  console.log("event:", obj)
  if (obj.event === 'add') {
    // 打开添加分类iframe
    layer.open({
      type: 2, //层类型, 默认0信息框, 1页面层, 2iframe层, 3加载层, 4tips层
      title: "添加分类", //标题
      content: "/html/add-category.html", //内容, 格式string/Array/DOM, 根据类型会改变
      area: ["400px", "175px"], //指定弹层宽高
      end: function() {
        table.reloadData('categoryTable');
        console.log("添加分类iframe关闭, 刷新数据已执行");
      }
    });
  } else if (obj.event === 'refresh') {
    table.reloadData('categoryTable');
  }
});
//编辑 或 删除分类项
table.on('tool(category-table)', function(obj) { // 双击 toolDouble
  var data = obj.data; // 获得当前行数据
  if (obj.event === 'edit') {
    // 打开修改分类iframe
    layer.open({
      type: 2,
      title: "修改分类",
      content: "/html/edit-category.html?id=" + data.id,
      area: ["400px", "225px"], //指定弹层宽高
      end: function() {
        table.reloadData('categoryTable');
        console.log("添加分类iframe关闭, 刷新数据已执行");
      }
    });
  } else if (obj.event === 'delete') {
    $.ajax({
      url: "/category/delete?id=" + data.id,
      type: 'DELETE',
      data: {},
      success: function(d) {
        if (d.code === 200) {
          table.reloadData('categoryTable');
        } else {
          layer.msg('删除失败, 原因: ' + d.msg);
        }
      }
    });
  }
});

```

图附录-39 分类管理 JS 代码 2

```

table.on('tool(borrow-table)', function(obj) { // 双击 toolDouble
    var data = obj.data; // 获得当前行数据
    if (obj.event === 'bookDetail') { //图书详情
        location.href = '/html/book-detail.html?id=' + data.bookId;
    } else if (obj.event === 'readDirect') { //立即阅读
        location.href = "/html/read.html?id=" + data.bookId;
    } else if (obj.event === 'earlyReturn') { //提前还书
        var index = layer.confirm('确定提前还书?', {
            btn: ['确定', '关闭'] //按钮
        }, function() {
            $.ajax({
                url: "/borrowReturn/return?bookId=" + data.bookId,
                type: 'POST',
                data: {},
                dataType: "json",
                contentType: "application/json",
                success: function(d) {
                    if (d.code === 200) {
                        table.reloadData('borrowTable');
                    } else {
                        layer.msg('还书失败, 原因: ' + d.msg);
                    }
                },
                error: function(e) {
                    layer.msg('还书失败, 网络可能没连通');
                }
            });
        });
    } else if (obj.event === 'borrowAgain') { //再次借阅
        $.ajax({
            url: "/borrowReturn/borrow?bookId=" + data.bookId,
            type: 'POST',
            data: {},
            dataType: "json",
            contentType: "application/json",
            success: function(d) {
                if (d.code === 200) {
                    layer.msg('借书5天成功, 要抓紧读哟~', {
                        icon: 1,
                        time: 1000 // 设置1秒后自动关闭
                    }, function() {
                        table.reloadData('borrowTable');
                    });
                } else {
                    layer.msg(d.msg);
                }
            },
            error: function(e) {
                layer.msg('借书失败, 网络可能没连通');
            }
        });
    }
});

```

图附录-40 借阅管理 JS 代码

```

//取消收藏
table.on('tool(collection-table)', function(obj) { // 双击 toolDouble
    var data = obj.data; // 获得当前行数据
    if (obj.event === 'delete') {
        // console.log(data)
        $.ajax({
            url: "/collection/cancel?bookId=" + data.bookId,
            type: 'DELETE',
            data: {},
            success: function(d) {
                if (d.code === 200) {
                    table.reloadData('collectionTable');
                } else {
                    layer.msg('取消失败, 原因: ' + d.msg);
                }
            },
            dataType: "json",
            contentType: "application/json"
        });
    } else if (obj.event === 'bookDetail') {
        location.href = '/html/book-detail.html?id=' + data.bookId;
    }
});
});

```

图附录-41 收藏管理 JS 代码

```

// 更新剩余时间循环
async function updateLastTimeLoop(stopTime) {
    const forceTime = new Date(Date.parse(stopTime));
    let now = new Date();
    let lastTime = forceTime.valueOf() - now.valueOf();
    console.log("updateLastTimeLoop lastTime=" + lastTime);
    let timeStr;
    if (lastTime <= 0) {
        updateLastTimeStr(timestamp2String(0))
    }
    while (lastTime >= 0) {
        timeStr = timestamp2String(lastTime);
        updateLastTimeStr(timeStr);
        await sleep(60 * 1000); // 每分钟更新一次
        lastTime -= 60 * 1000;
    }
}
//获取请求路径中的图书ID
function getBookId() {
    var strArray = location.href.split("?id=");
    return strArray[1];
}
window.onload = function() {
    let newIframe = document.createElement("iframe");
    newIframe.setAttribute("id", "pdf-frame");
    var $ = layui.$;
    $.ajax({
        url: "/borrowReturn/read?id=" + getBookId(),
        type: "GET",
        success: function(data) {
            if (data.code === 200) {
                var json = data.data;
                console.log(json)
                // 替换图书封面
                let bookCover = document.getElementById("book-cover");
                bookCover.src = json.book.cover;
                // 动态替换pdf
                let pdfUrl = json.book.pdfUrl;
                newIframe.src = pdfUrl === null ? '' : pdfUrl;
                //书名与作者替换
                $("#bookName").text("《" + json.book.name + "》");
                $("#bookAuthor").text(json.book.author);
                // pdf阅读
                newIframe.setAttribute("frameborder", "0");
                let container = document.querySelector(".container");
                container.appendChild(newIframe);
                //js计时到期, 提示, 去除iframe中的src
                calculateStopWatch(newIframe, json.forceReturnTimeStr);
                updateLastTimeLoop(json.forceReturnTimeStr)
            } else {
                alert("登录已过期, 关闭窗口自动跳转到首页。")
                location.href = "/index.html";
            }
        }
    });
}

```

图附录-42 在线阅读 JS 代码